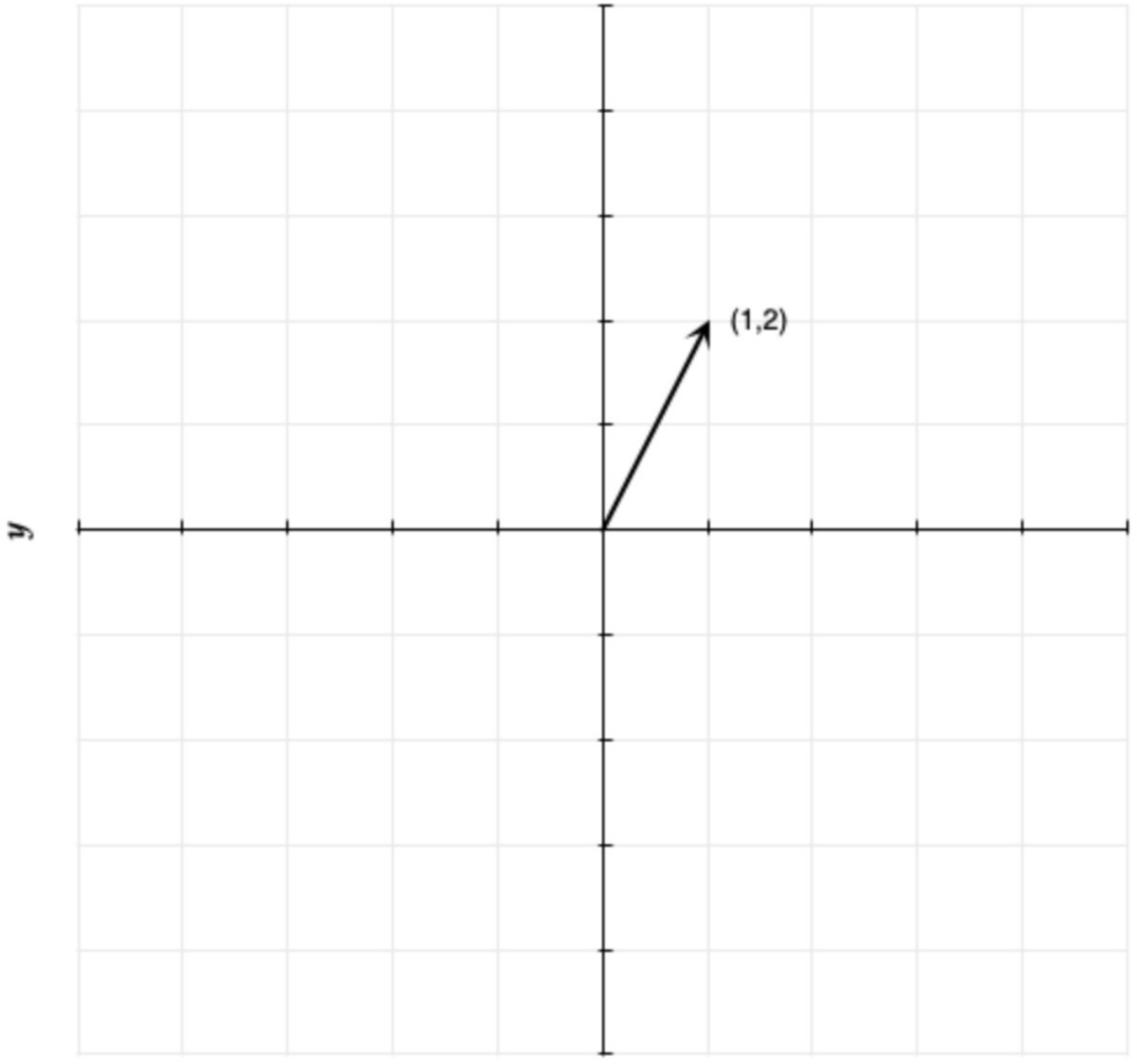
Language Models

From Vectors to ChatGPT

Vectors



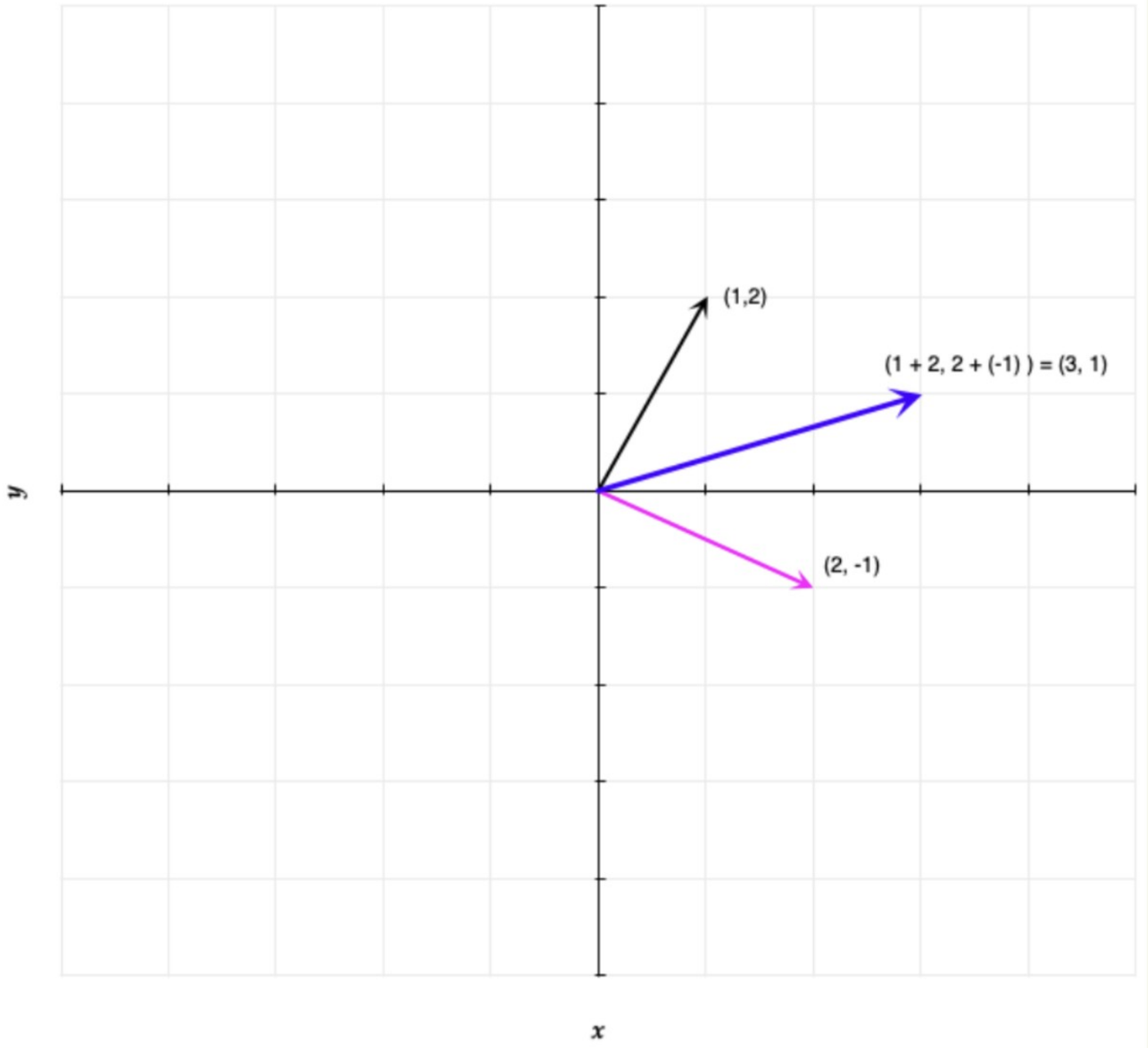
Geometric Interpretation of a vector

Vectors are arrows pointing in space.

For $\vec{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, imagine moving 1 step to the right (x-axis) and 2 steps up (y-axis) from the origin.

Every pair of numbers corresponds to one arrow, and every such arrow from the origin, corresponds to a unique pair of numbers.

Thus, a vector has a visual *length* and *direction* in space.

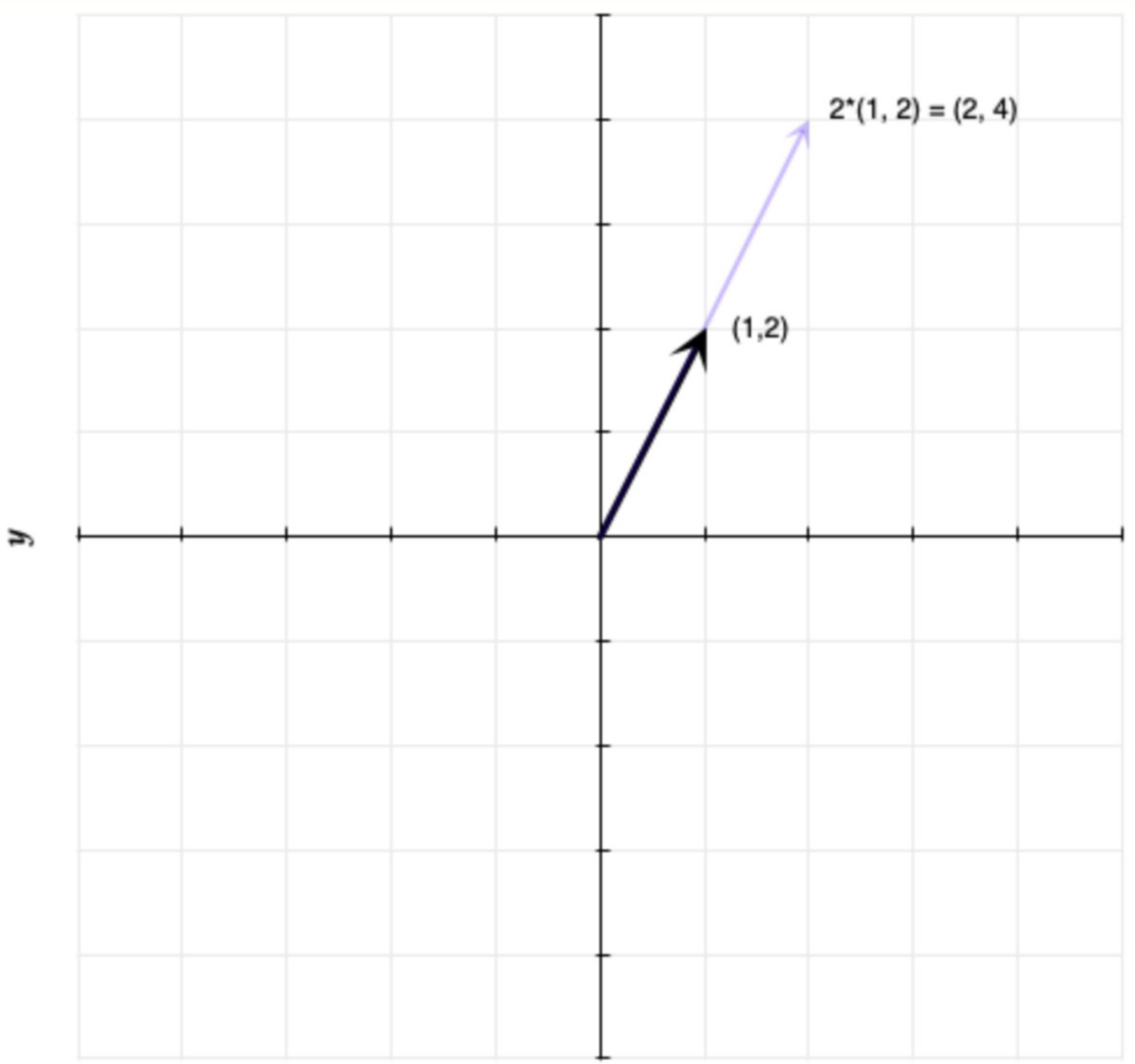


Vector addition

We can add vectors by adding the individual components using the geometric interpretation from previous. Here we first move one step right and 2 steps up, and then 2 steps right and one step *down* (-1). The net is 3 steps right and 1 step up. This can be seen as:

$$\vec{u} = \vec{v} + \vec{w} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} 1+2 \\ 2+(-1) \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$$

We are adding the x components in the column vectors to get the x component of the new column vector $\vec{u}$. We do the same for the y components.



Scalar multiplication

This is the idea of multiplying a vector by a number, $\lambda\vec{v}$ for some number λ, so called because it *scales* the vector by whatever number you multiplied the vector by.

Here

$$\vec{u} = 2\vec{v} = 2 \times \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 \times 1 \\ 2 \times 2 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \end{pmatrix}.$$

The geometric interpretation is to walk twice as long in the same direction.
Thecorresponding mathematical idea is to multiply each component by the same *scalar* so that we *scale* the vector.

Dot Products

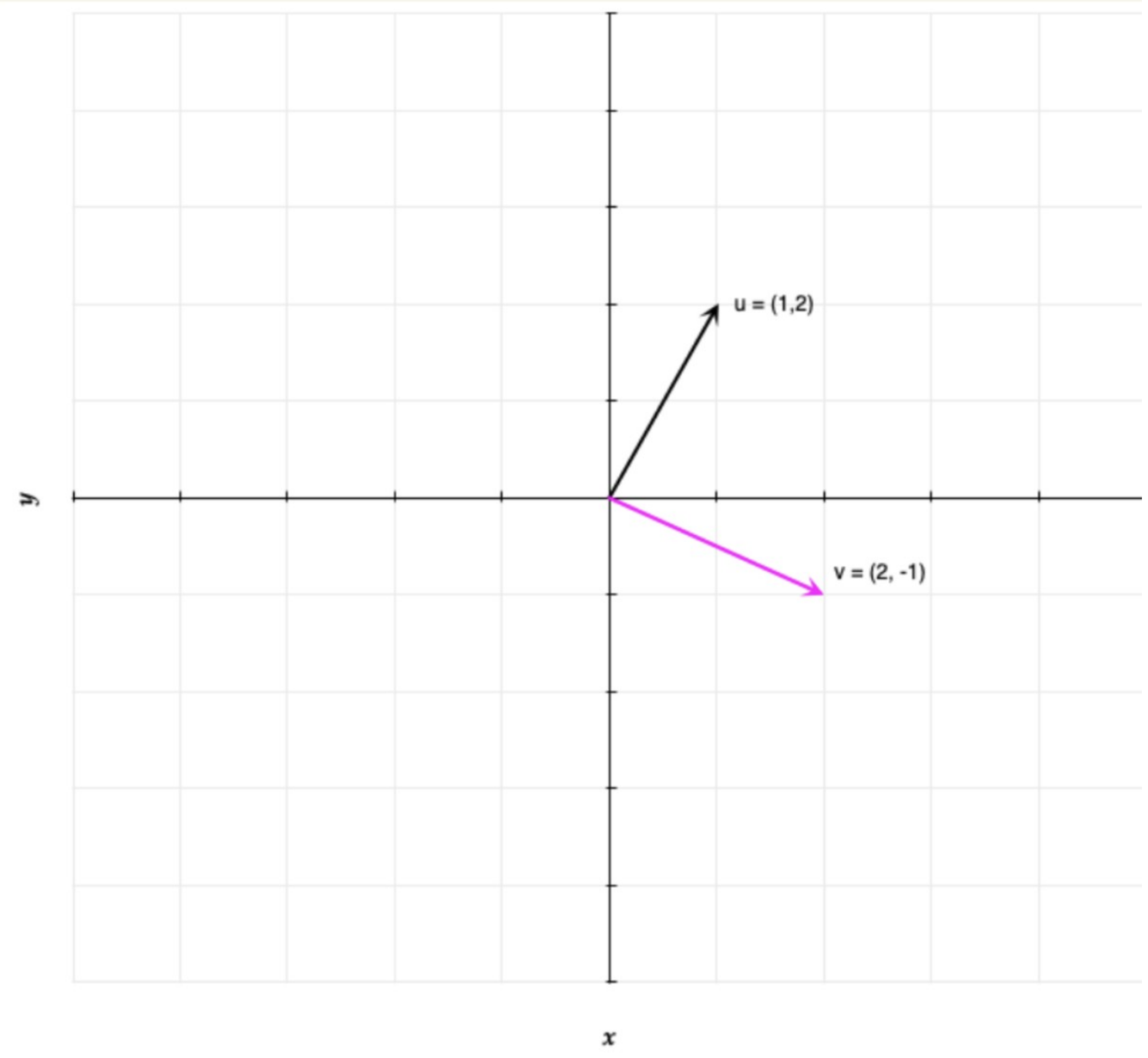
The dot product of two vectors $\vec{u}$ and $\vec{v}$ is defined as the sum of the products of the vectors components:

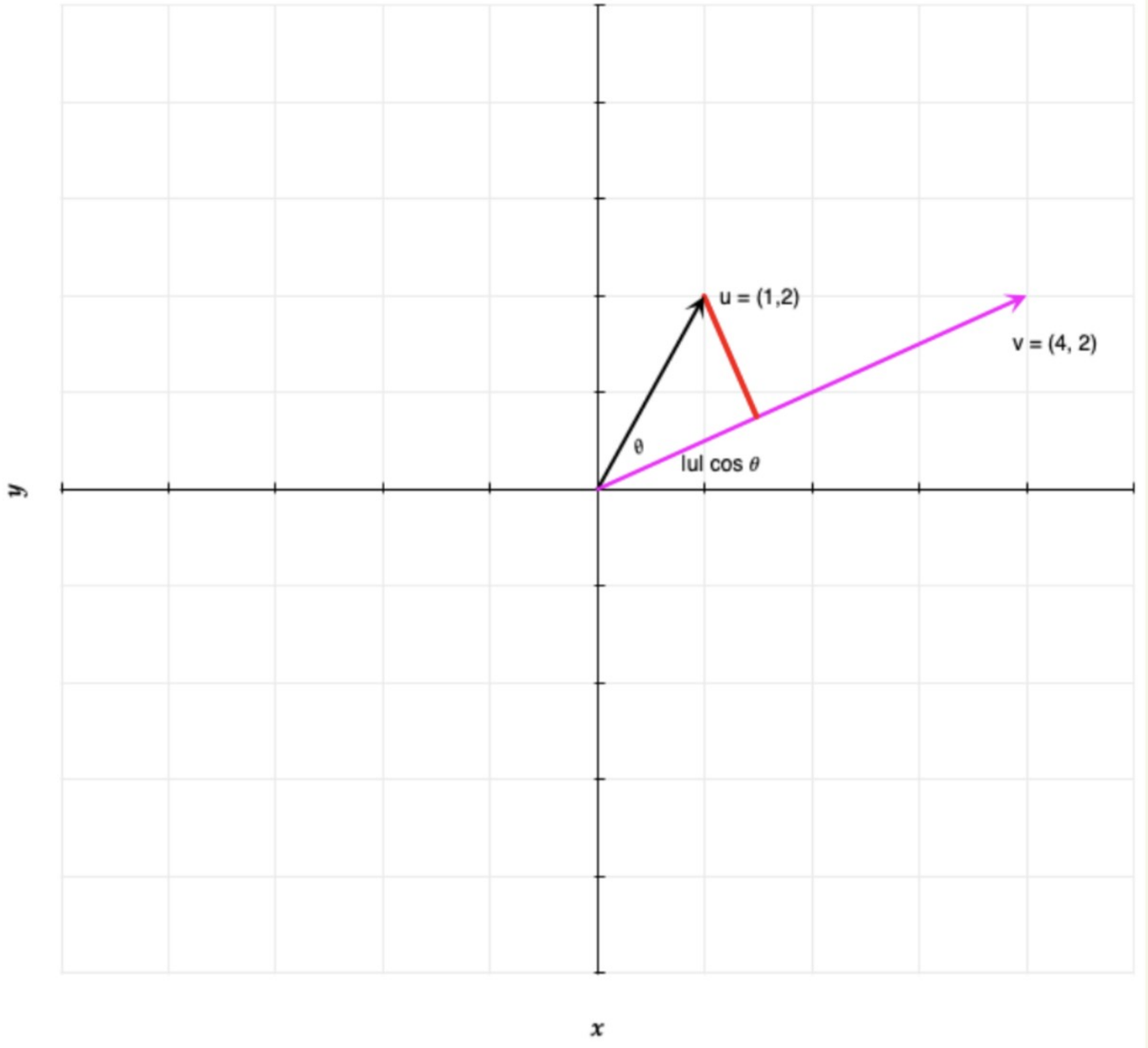
$$\vec{u} \cdot \vec{v} = \begin{pmatrix} u_x \\ u_y \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \end{pmatrix} = u_x v_x + u_y v_y$$

In this case, we have:

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ -1 \end{pmatrix} = 1 \times 2 + 2 \times -1 = 0$$

These vectors are *perpendicular* or *orthogonal*: their dot product is 0.





You might recall a definition from high school:

The dot product of two vectors $\vec{u}$ and $\vec{v}$ is defined as the product of $\vec{v}$ with the projection of $\vec{u}$ onto $\vec{v}$

$$\vec{u} \cdot \vec{v} = |\vec{v}| \times |\vec{u}| \cos\theta,$$

where $|\vec{v}|$ is the length of $\vec{v}$ and θ the angle between $\vec{u}$ and $\vec{v}$.

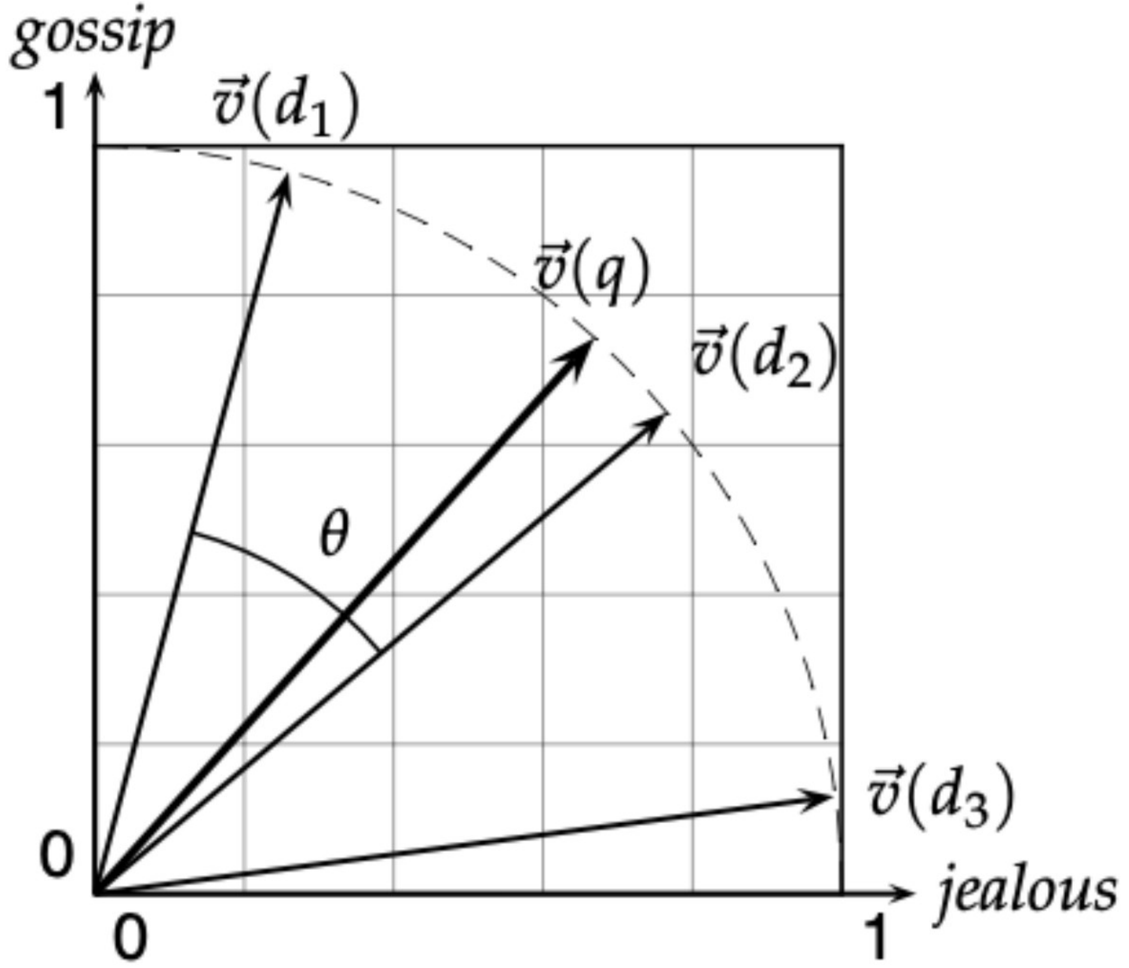
These definitions are identical. Imagine a new x-axis aligned with the pink vector. Then the product becomes simply $u_x v_x$ and is equivalent to the previous definition. You can also prove this (with more effort) in the general case.

Cosine Similarity

The cosine similarity is simply the dot product of two vectors divided by their magnitudes.

$$\text{sim}(d_1, d_2) = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{\left|\vec{V}(d_1)\right| \left|\vec{V}(d_2)\right|'}$$

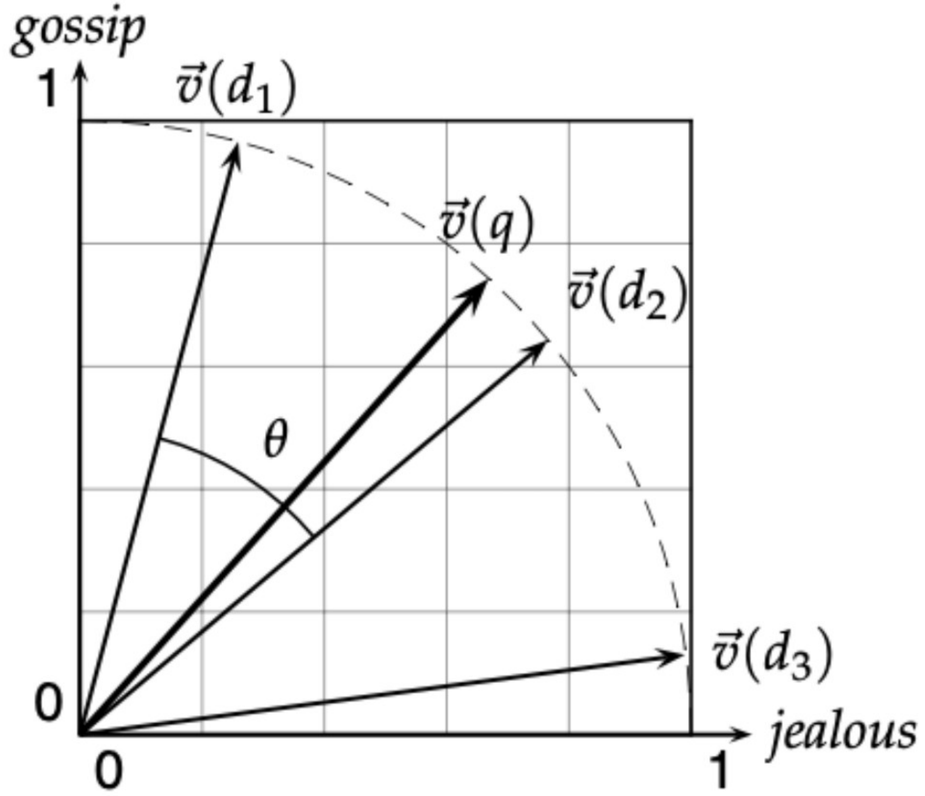
term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6



cosine similarity:

cosine of the angle between two unit vectors

term	SaS	PaP	WH
affection	0.996	0.993	0.847
jealous	0.087	0.120	0.466
gossip	0.017	0	0.254



It can be used to make queries:

$$\text{score}(q,d) = \frac{\vec{V}(q)\cdot\vec{V}(d)}{|\vec{V}(q)||\vec{V}(d)|}$$

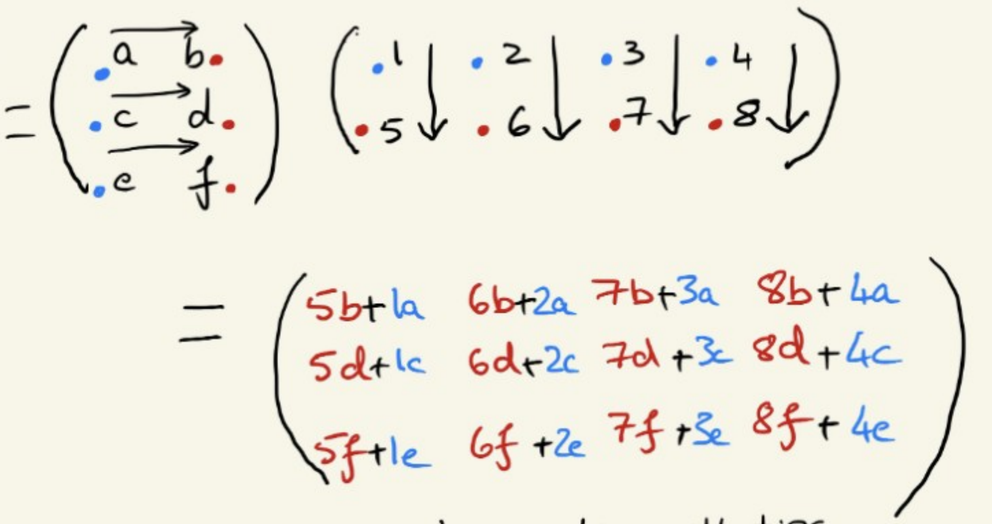
Consider the query $q =$ `jealous gossip`. This query turns into the unit vector $\vec{v}(q) = (0, 0.707, 0.707)$. Now assign to each document d a score equal to the dot product $\vec{v}(q)\cdot\vec{v}(d)$. Sort.

Matrix multiplication Recap

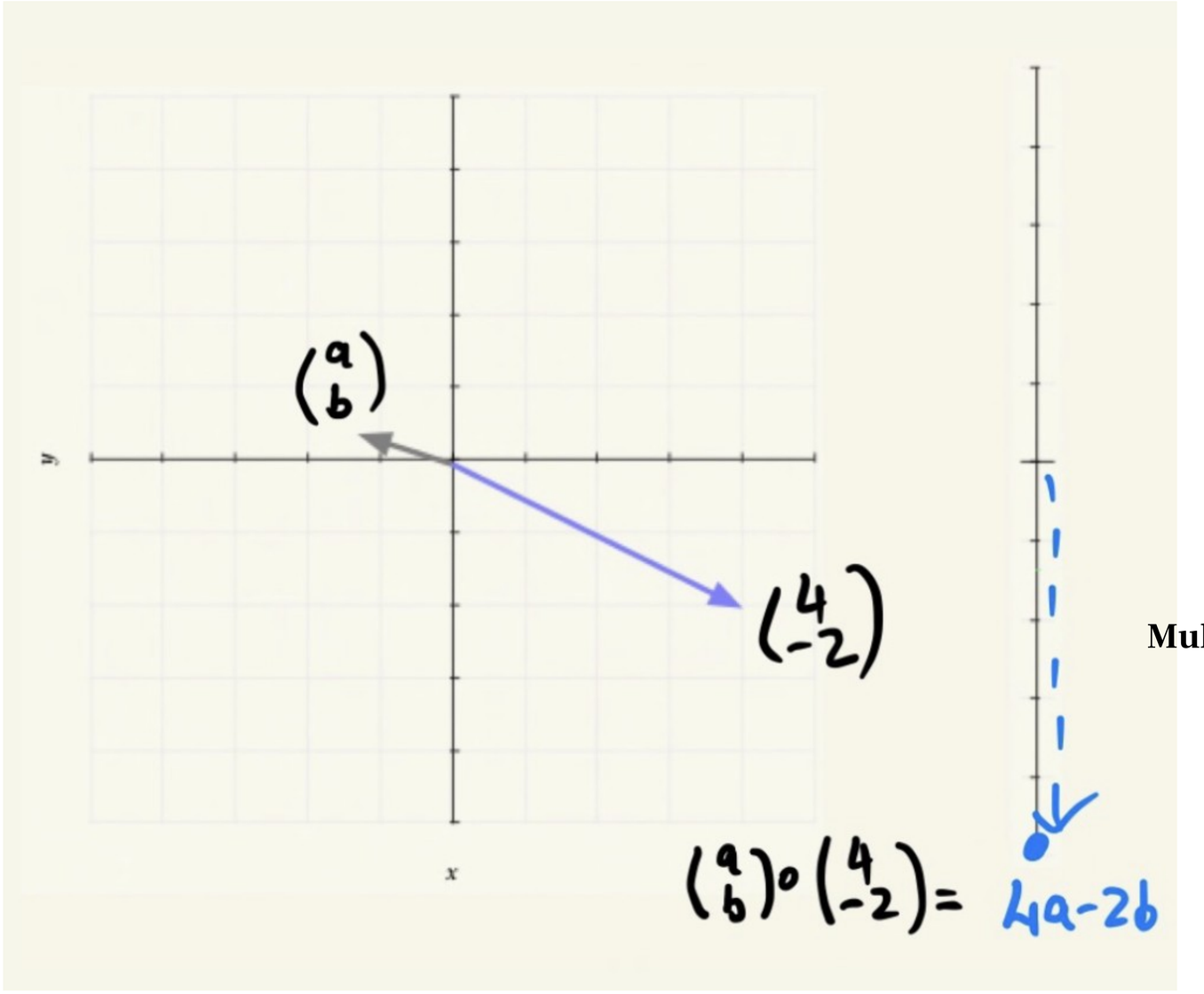
A and B are compatible for multiplication since their *inner dimensions* match:

$$3 \times 2, 2 \times 4$$

$$C = \begin{pmatrix} a & b \\ c & d \\ e & f \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{pmatrix}.$$



The product matrix $C = A \cdot B$ has the *outer dimensions*, thus $dim(C) = 3 \times 4$



$$T(\overline{v}) = \begin{pmatrix} a \\ b \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a & b \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = a\,x + b\,y$$

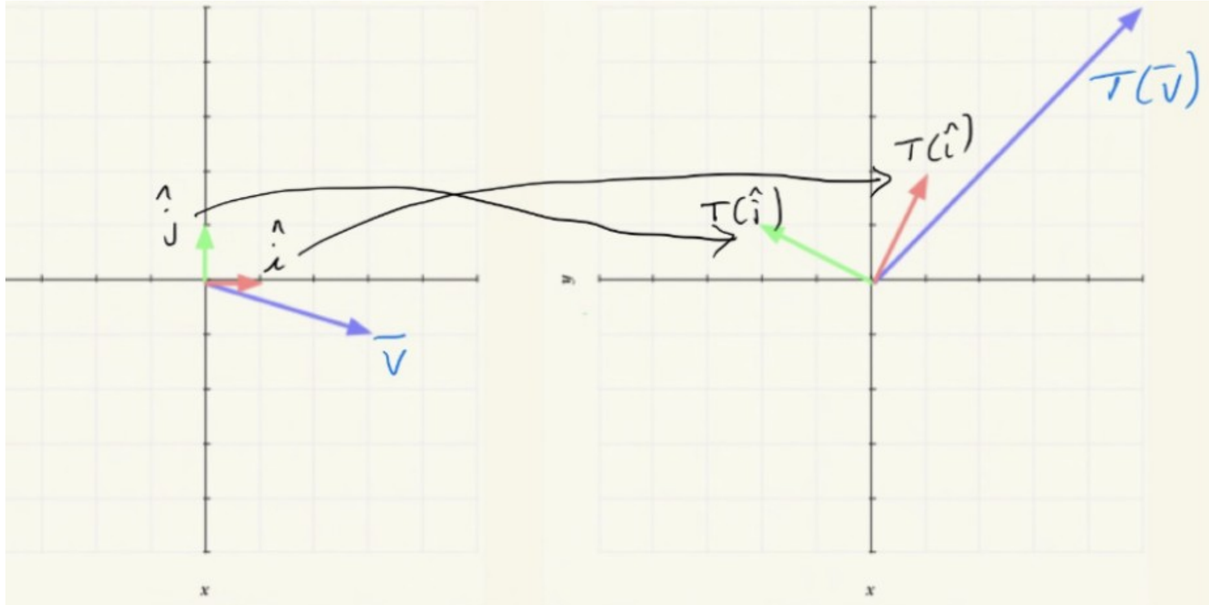
Dot products are a transform from 2D to 1D.

And can be thought of a very special kind of matrix multiplication

Multiple dot products can transform to other dimensions

Two ways to see matrix multiplication

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} (a & b)\begin{pmatrix} x \\ y \end{pmatrix} \\ (a & b)\begin{pmatrix} x \\ y \end{pmatrix} \end{pmatrix} = \begin{pmatrix} ax + by \\ cx + dy \end{pmatrix}$$



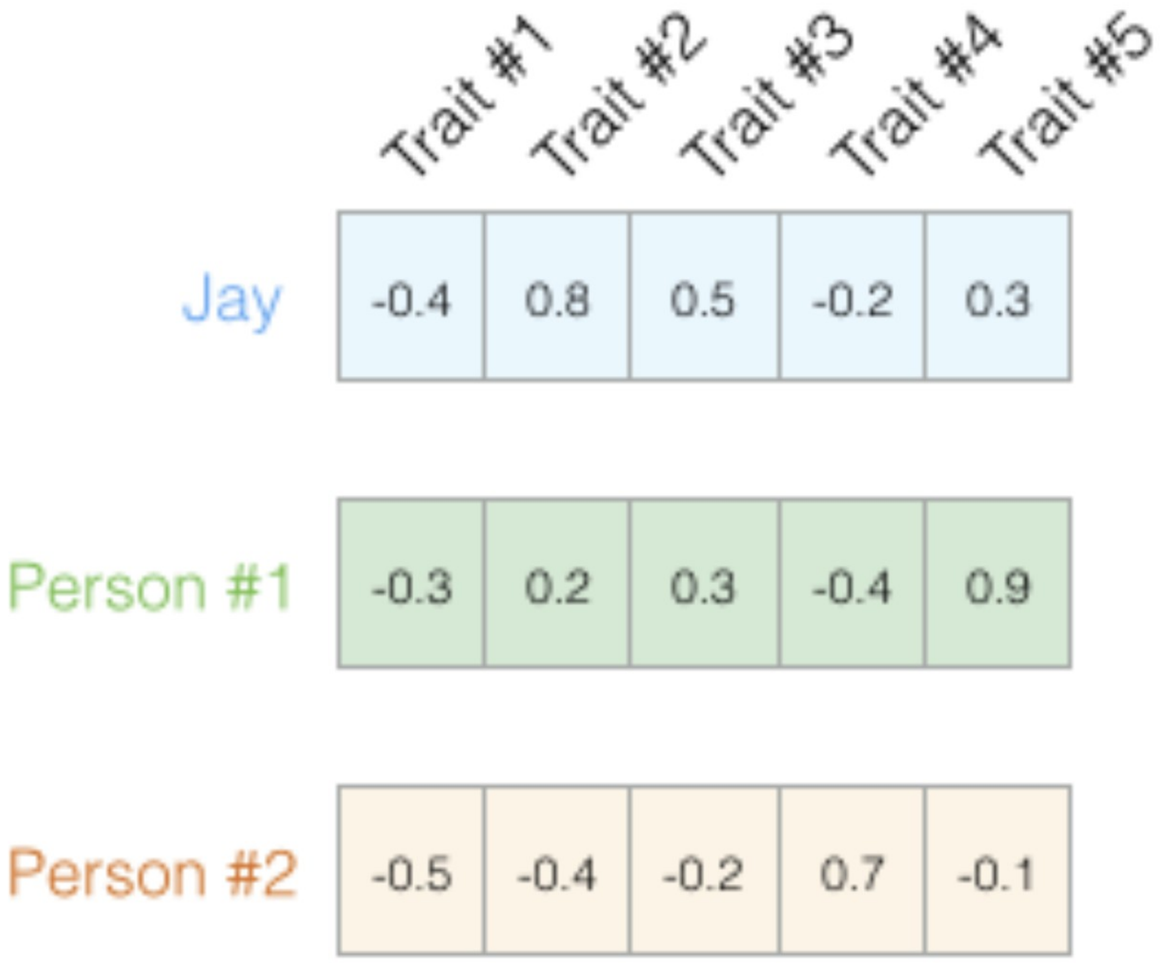
$$T\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = x\begin{pmatrix} a \\ c \end{pmatrix} + y\begin{pmatrix} b \\ d \end{pmatrix}$$

Scale the transformed unit vectors by the initial vector's co-ordinates and add. In our example:

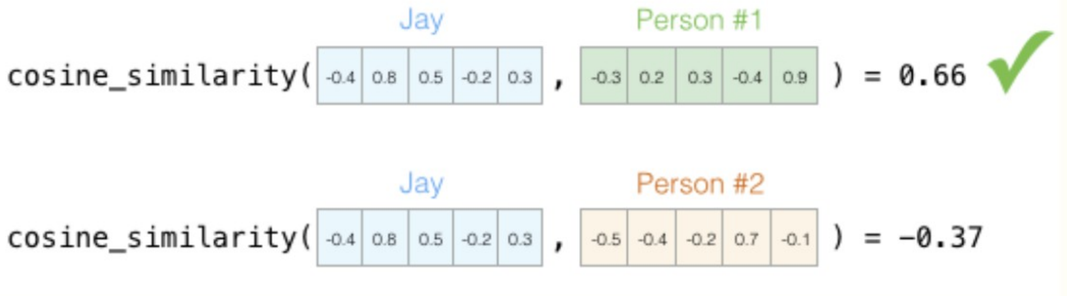
$$T\begin{pmatrix} 3 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 & -2 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \end{pmatrix} = 3\begin{pmatrix} 1 \\ 2 \end{pmatrix} - 1\begin{pmatrix} -2 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

Embeddings

The Basic Idea



- Observe a bunch of people
- **Infer** Personality traits from them
- Vector of traits is called an **Embedding**
- Who is more similar? Jay and who?
- Use Cosine Similarity of the vectors



Categorical Data

Example:

Rossmann Kaggle Competition. Rossmann is a 3000 store European Drug Store Chain. The idea is to predict sales 6 weeks in advance.

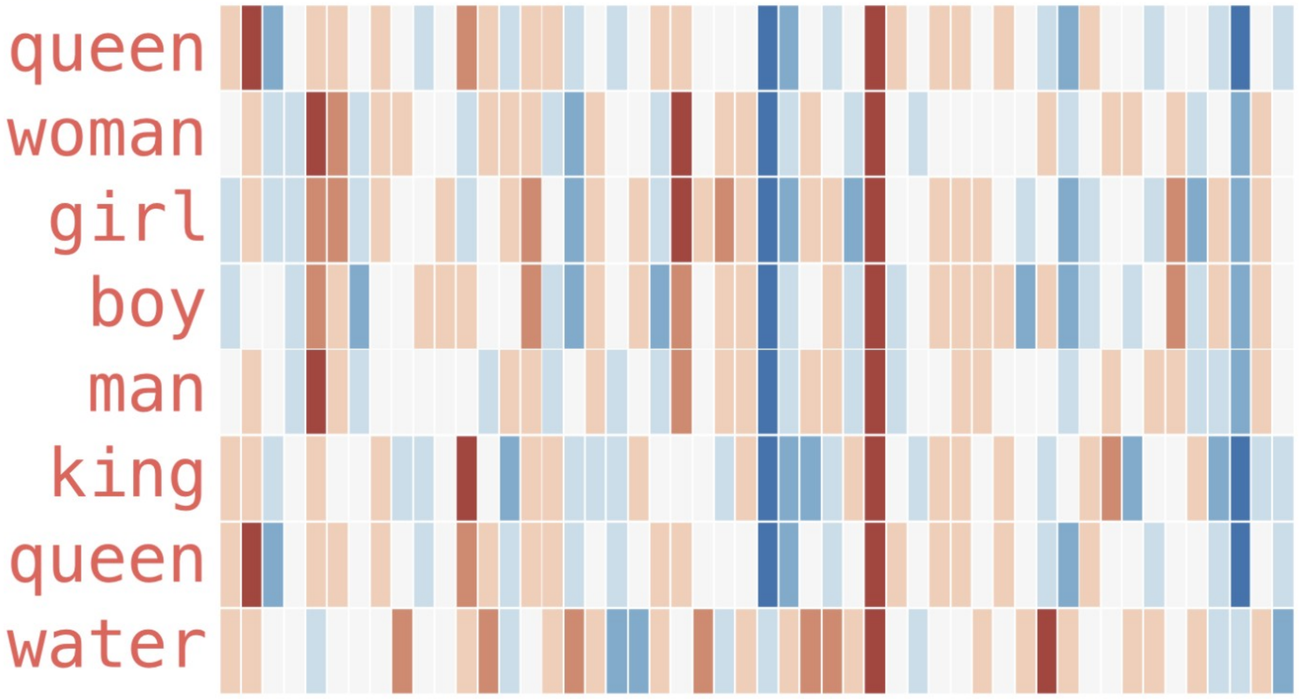
Consider `store_id` as an example. This is a **categorical** predictor, i.e. values come from a finite set.

We usually **one-hot encode** this: a single store is a length 3000 bit-vector with one bit flipped on.

What is the problem with this?

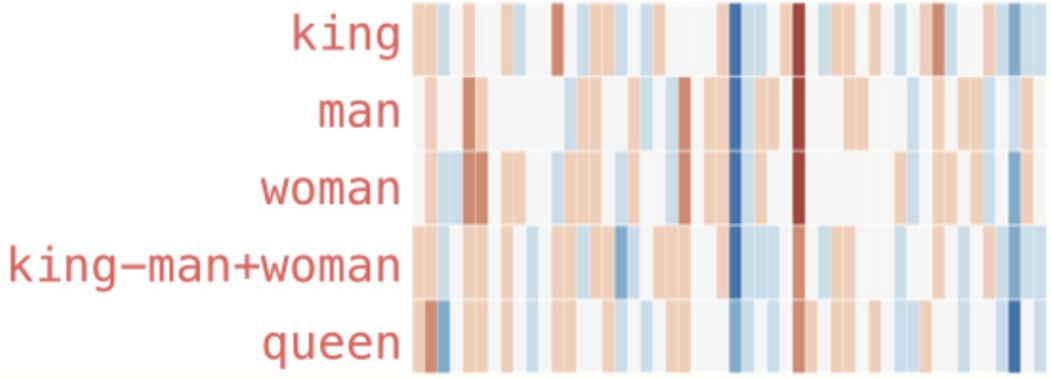
- The 3000 stores have commonalities, but the one-hot encoding does not represent this
- Indeed the dot-product (cosine similarity) of any-2 1-hot bitmaps must be 0
- Would be useful to learn a lower-dimensional **embedding** for the purpose of sales prediction.
- These store "personalities" could then be used in other models (different from the model used to learn the embedding) for sales prediction
- The embedding can be also used for other **tasks**, such as employee turnover prediction

Obligatory example

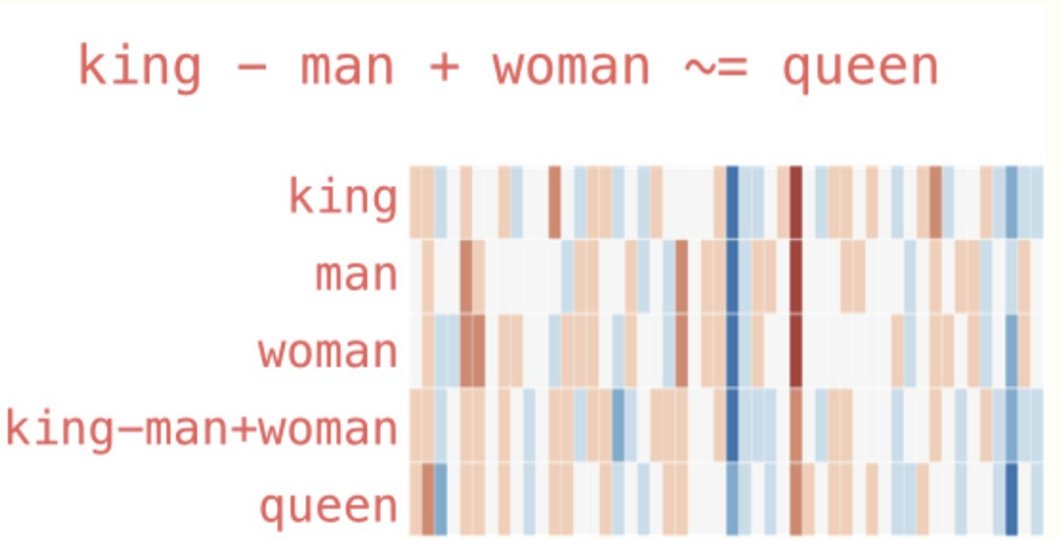


See man->boy as woman->girl, similarities of king and queen, for eg.
These are lower dimensional GloVe embedding vectors

king − man + woman ~= queen



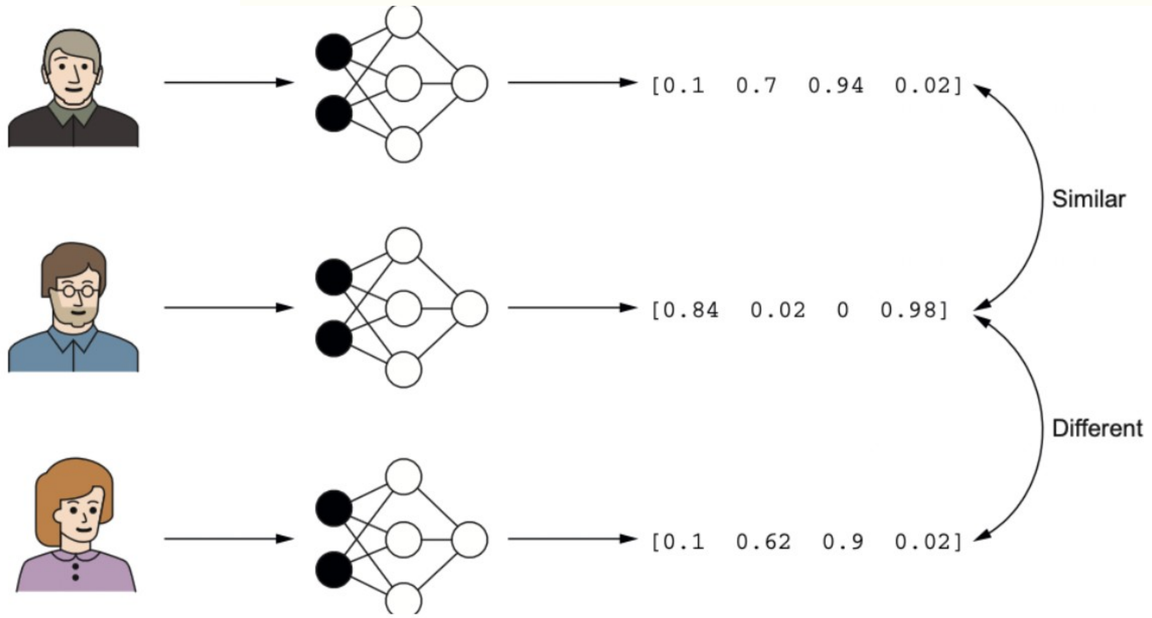




Embeddings map similar concepts to similar vectors

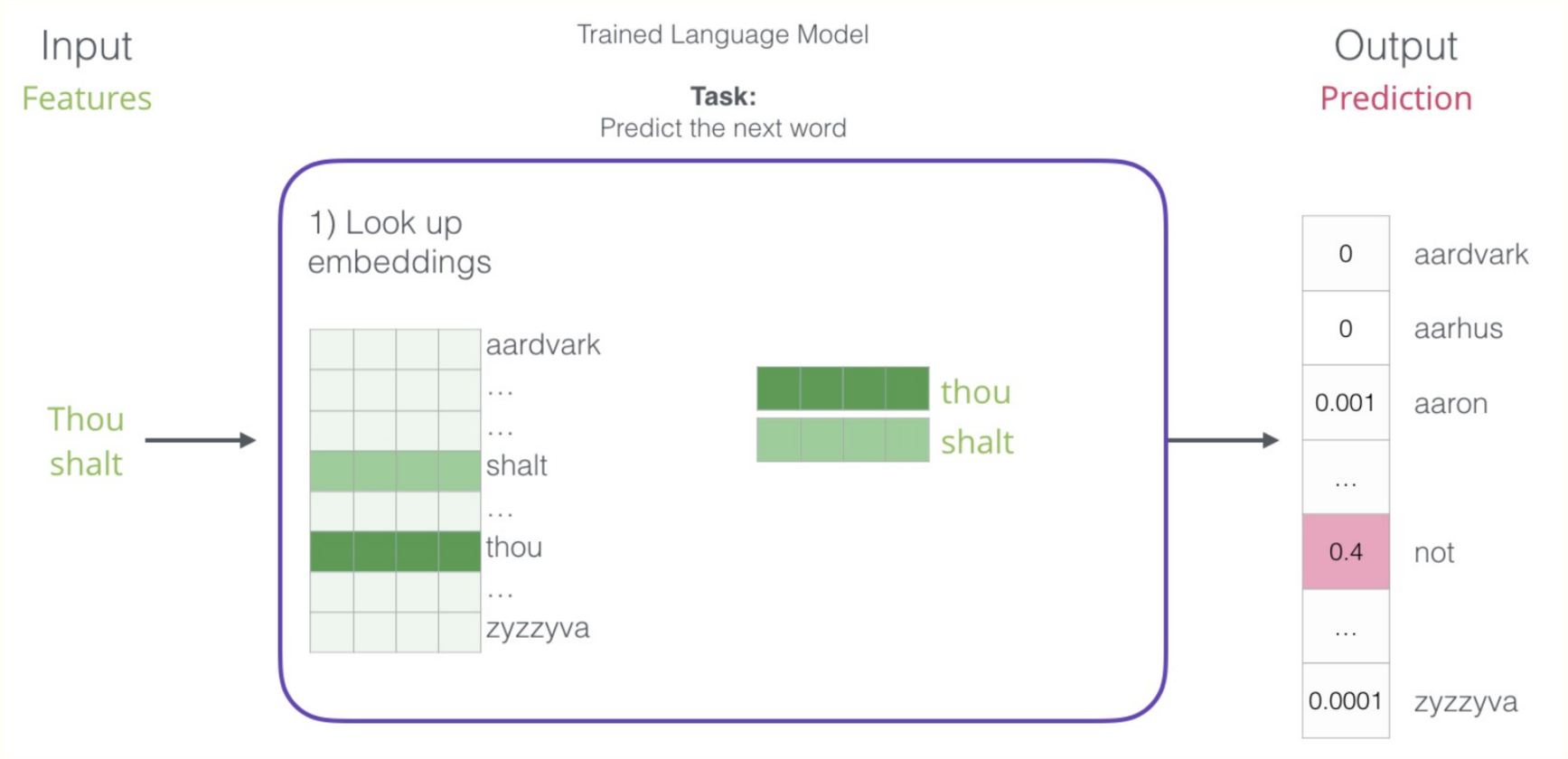
These concepts can be from different spaces, like users and movies

Any model can be used to produce embeddings: Apple face recognition.



How do we train word embeddings?

We need to choose a downstream task. We could choose **Language Modeling**: predict the next word. We'll start with random "weights" for the embeddings and other parameters and run SGD. A trained model+embeddings would look like this:



Why is language modeling a great task?

How do we set up a training set? All the labels are provided for us!!! This is **self-supervised** learning!

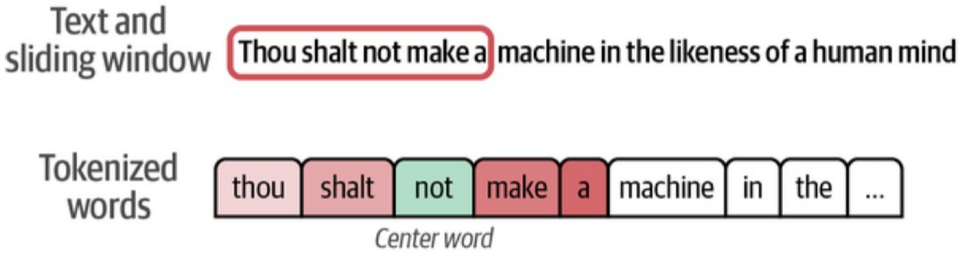
Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	

Dataset

input 1	input 2	output
thou	shalt	not
shalt	not	make
not	make	a
make	a	machine
a	machine	in

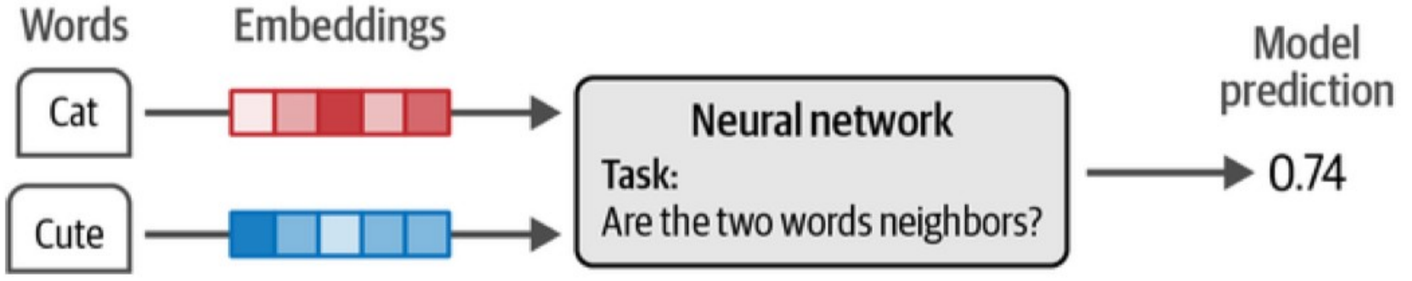


Word 1	Word 2	Target
not	thou	1
not	shalt	1
not	make	1
not	a	1
thou	apothecary	0
not	sublime	0
make	def	0
a	playback	0

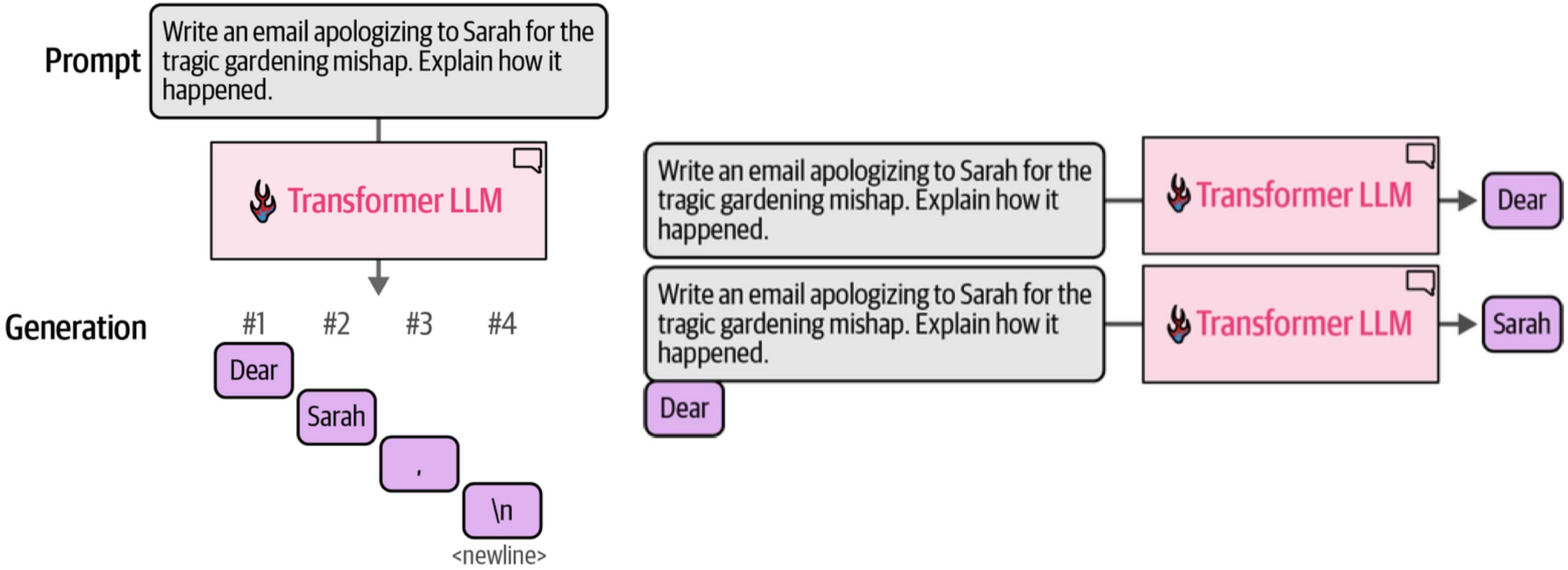
Positive examples

Negative examples

It is not enough to have just positive samples, you need negative samples too. These can be obtained by random sampling.

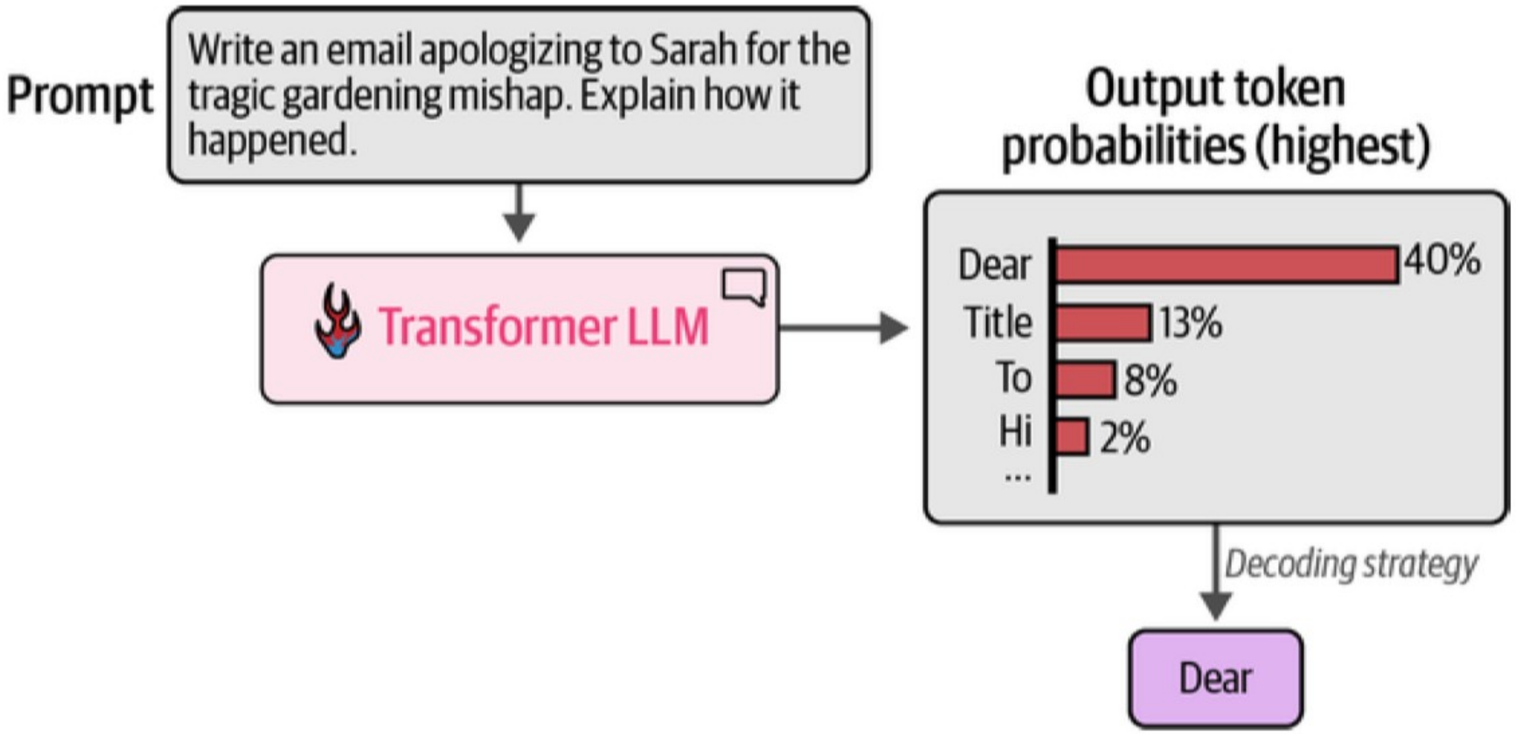


Generation happens token by token



Probabilities drive everything

We sample from probabilities to get the next word

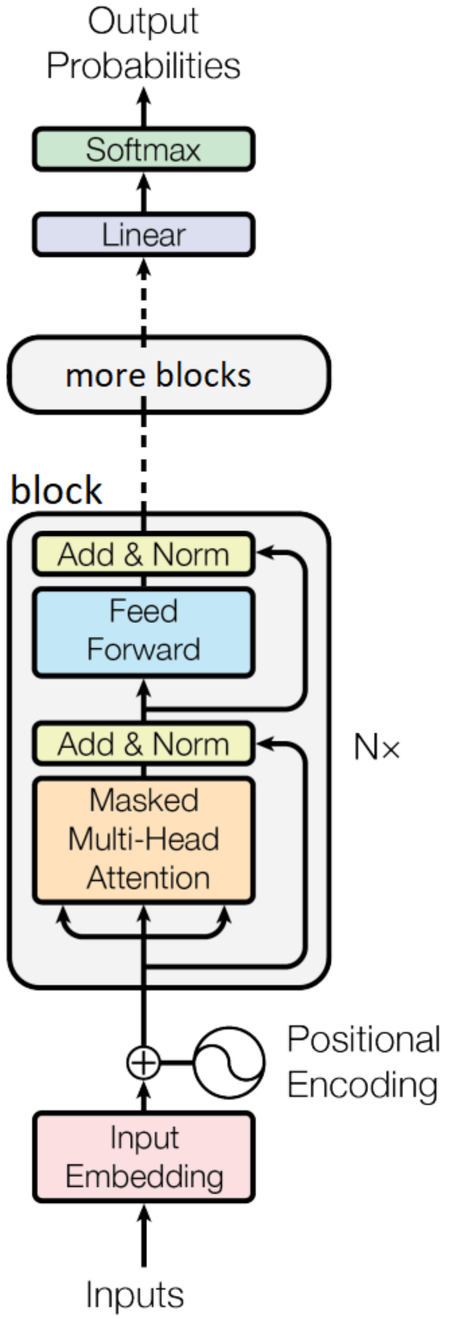


Why use a transformer architecture?

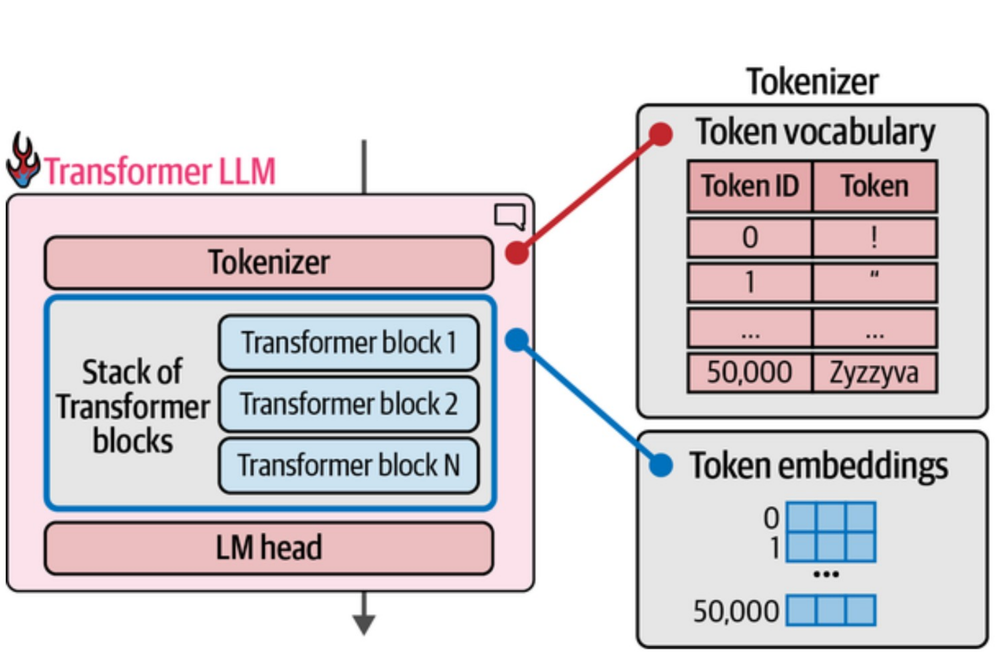
- **ATTENTION**: The transformer has a mechanism to figure out which words in a context are important to understanding the meaning of a given word and thus impacting its embedding

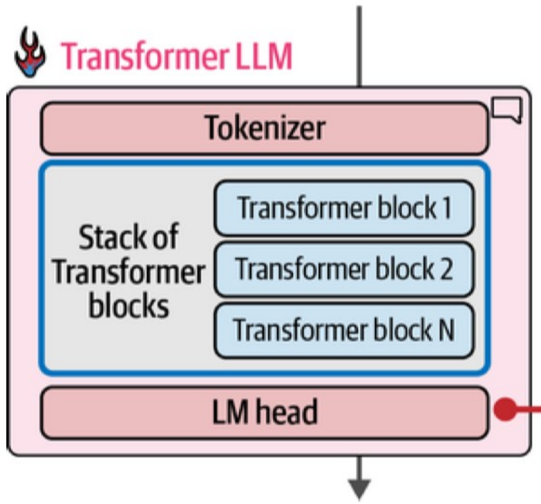
- **MEMORY**: The transformer has a MLP mechanism to associate with a word and its embedding facts that are pulled out from a kind of memory database.

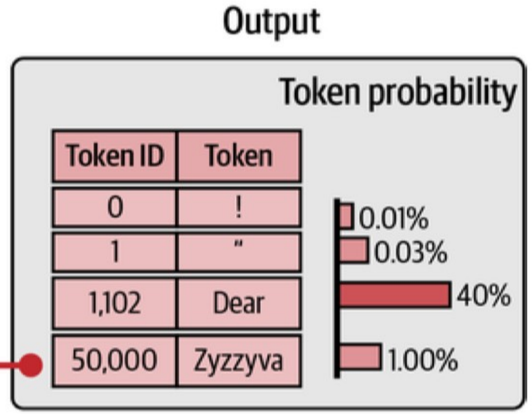
- **PARALLELISM**: At the level of parallel paths through tokens, parallel attention heads, and parallel paths through the memory bank (MLP) there are multiple opportunities to exploit the parallelism inherent in GPUs. This is in contrast to RNN mechanisms.

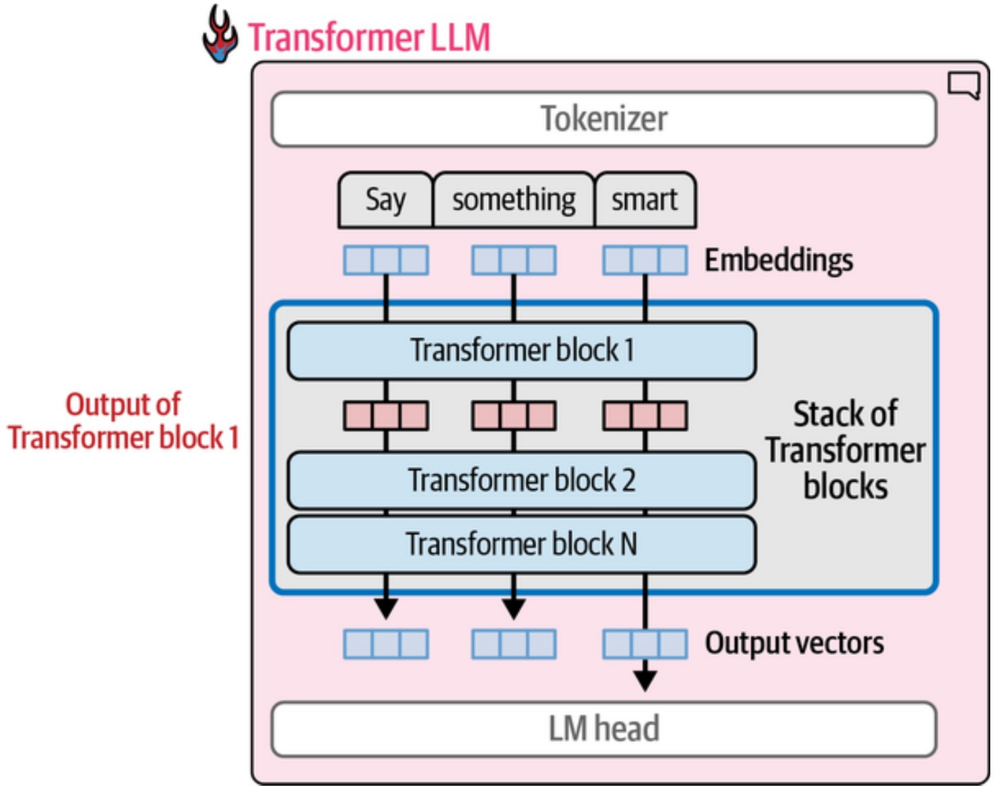


The GPT Architecture



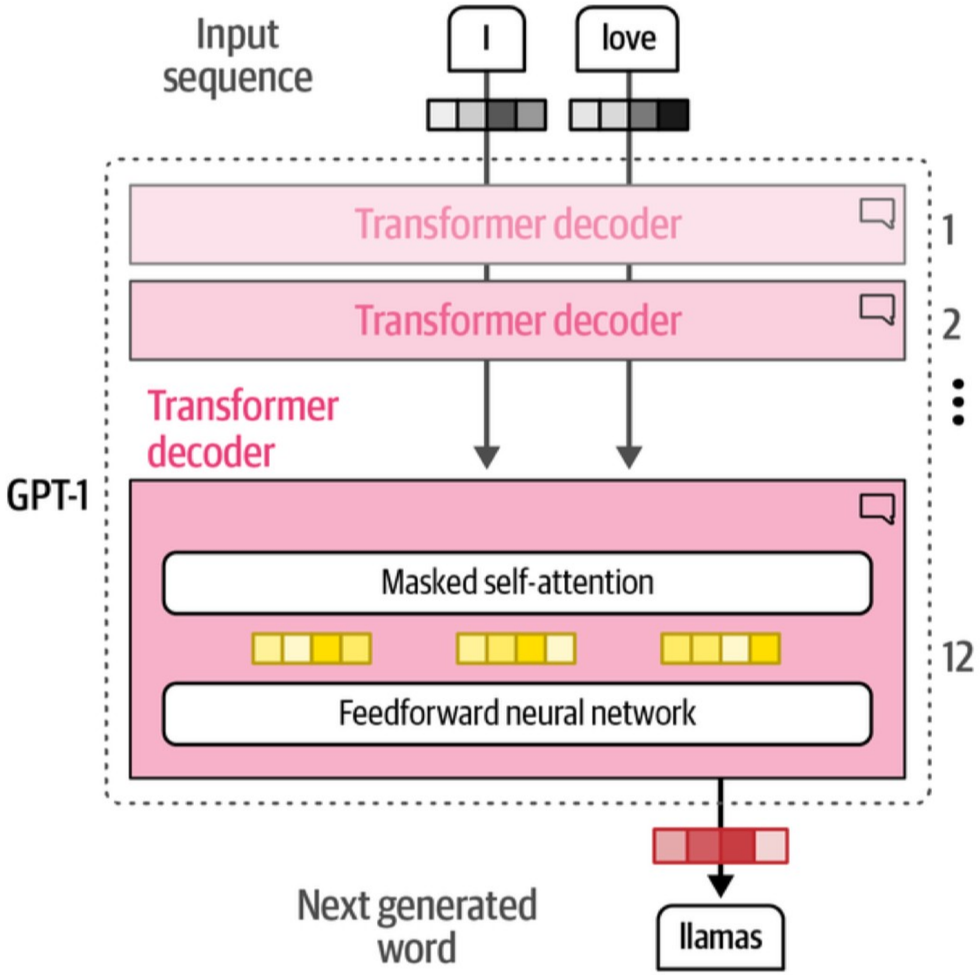


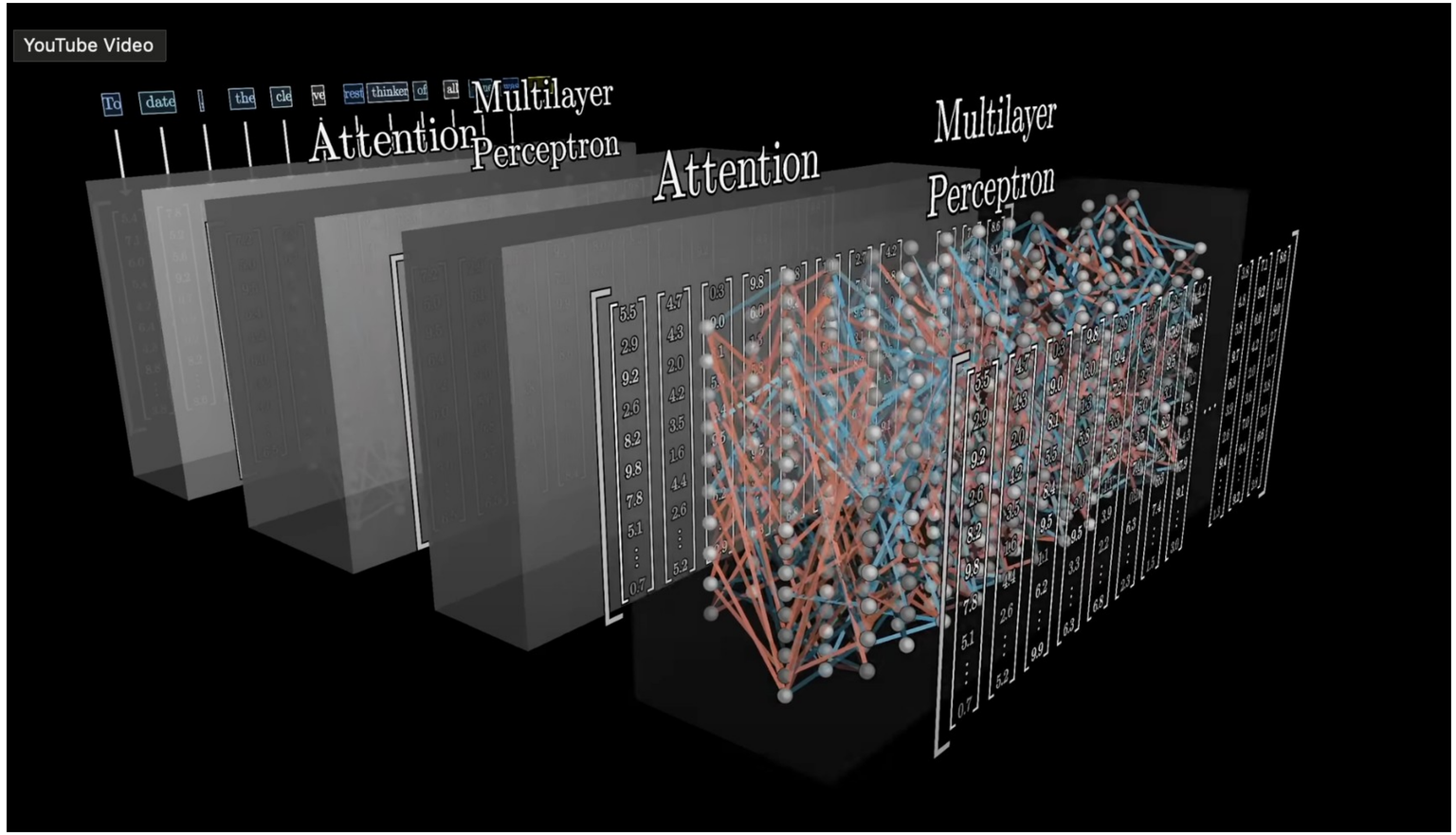


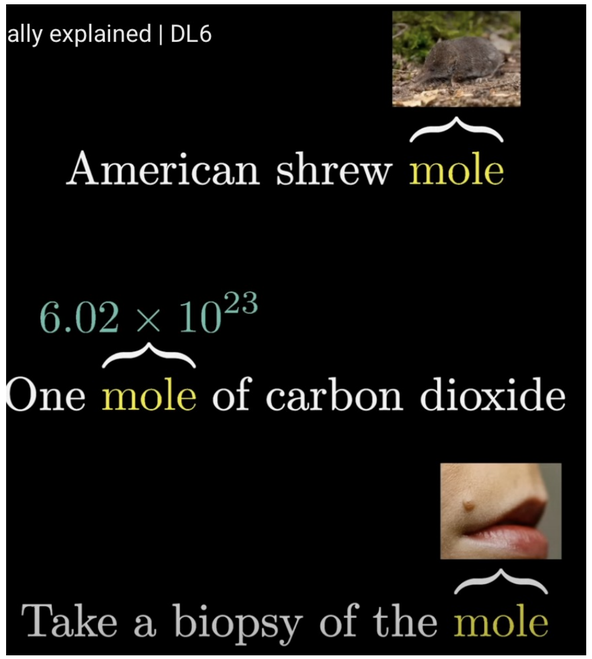


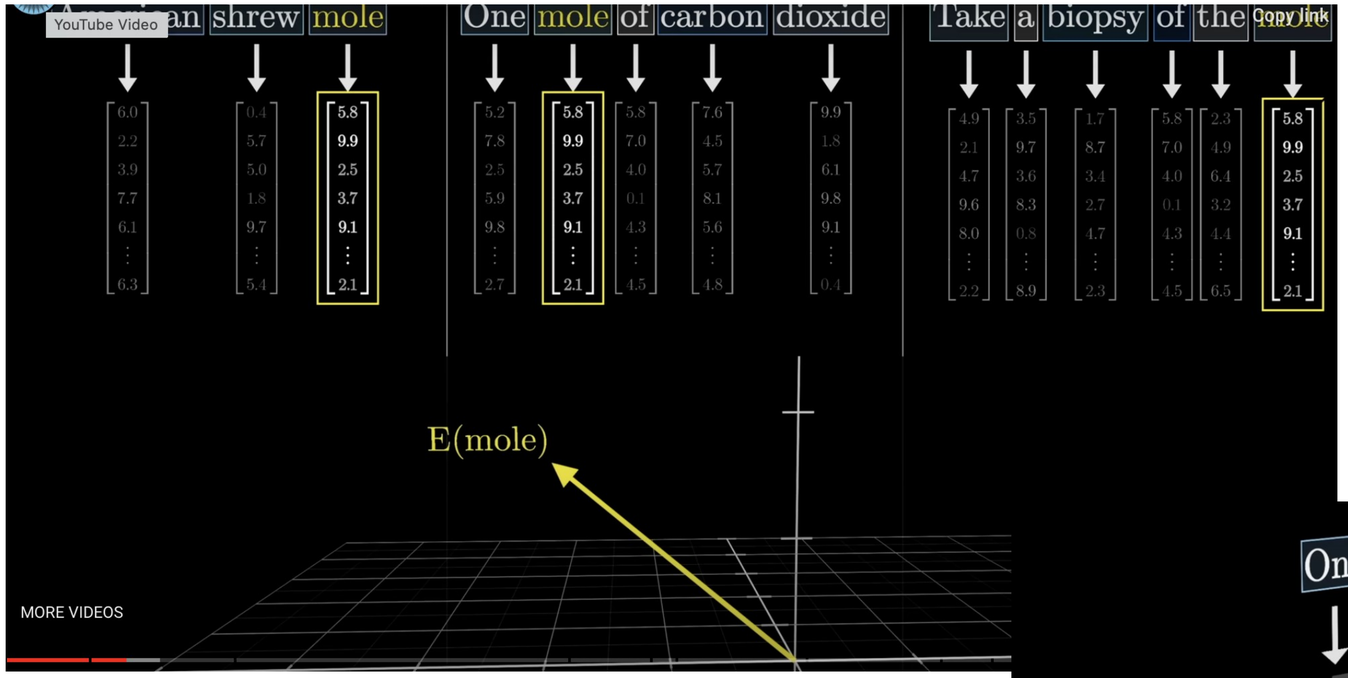
Output of Transformer block 1

What does the Transformer Block look like?



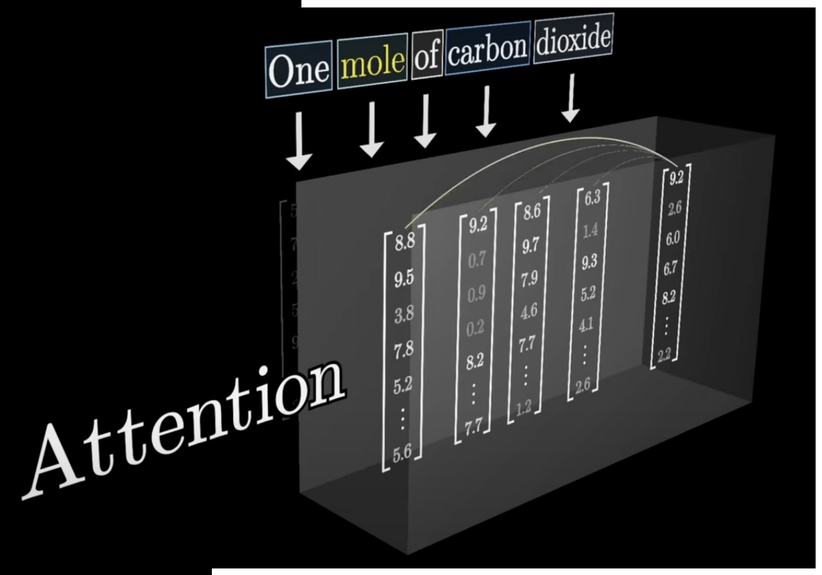


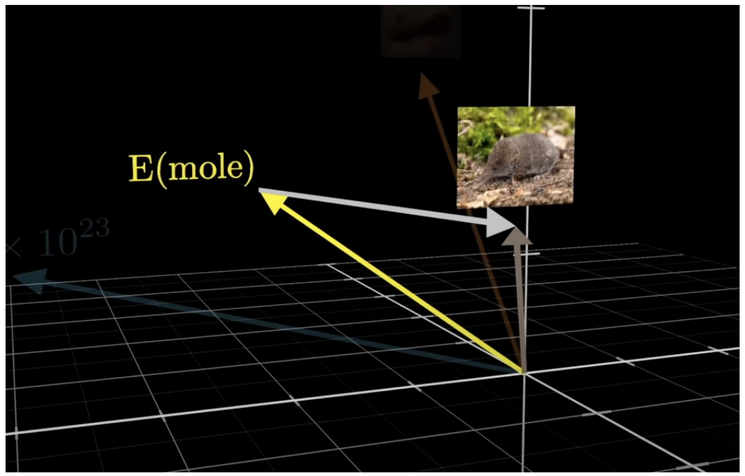


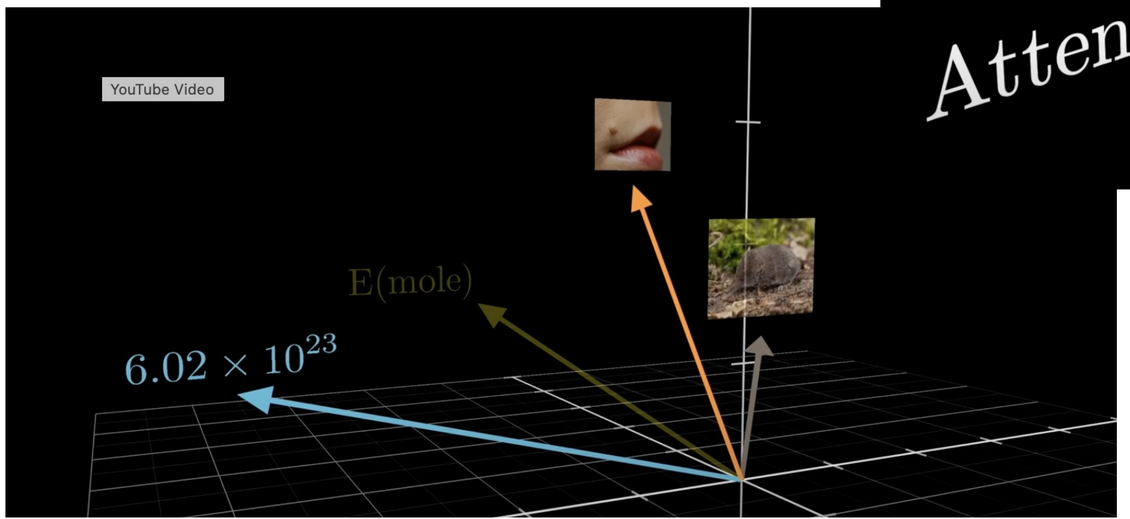


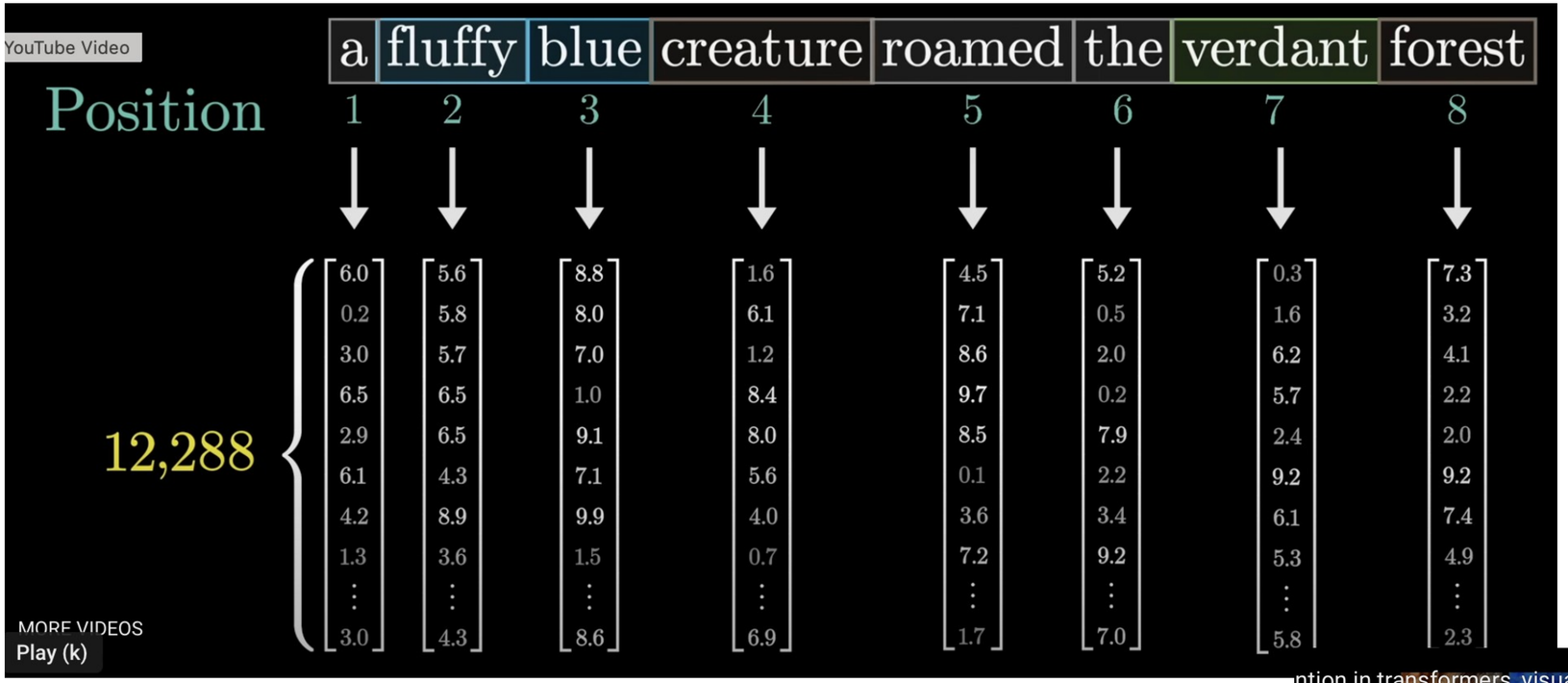
Attention Contextualizes

We start with un-contextualized embeddings for mole. The attention process produces vectors to add to the original mole embedding to contextualize them from their surrounding context.

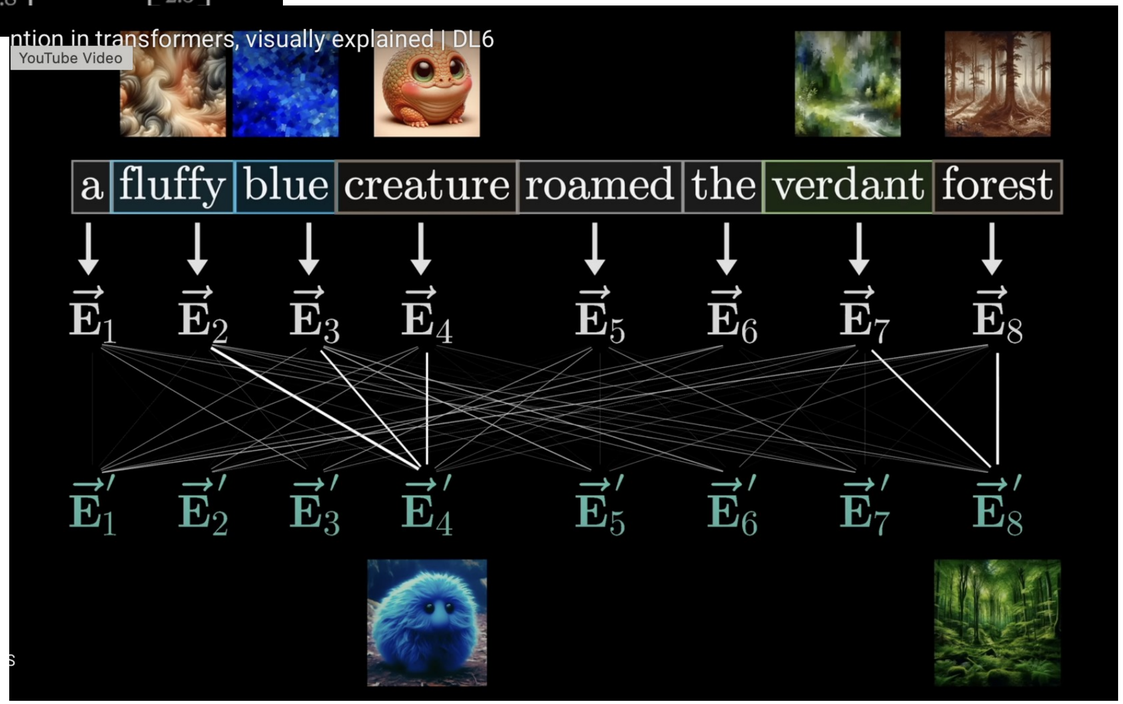


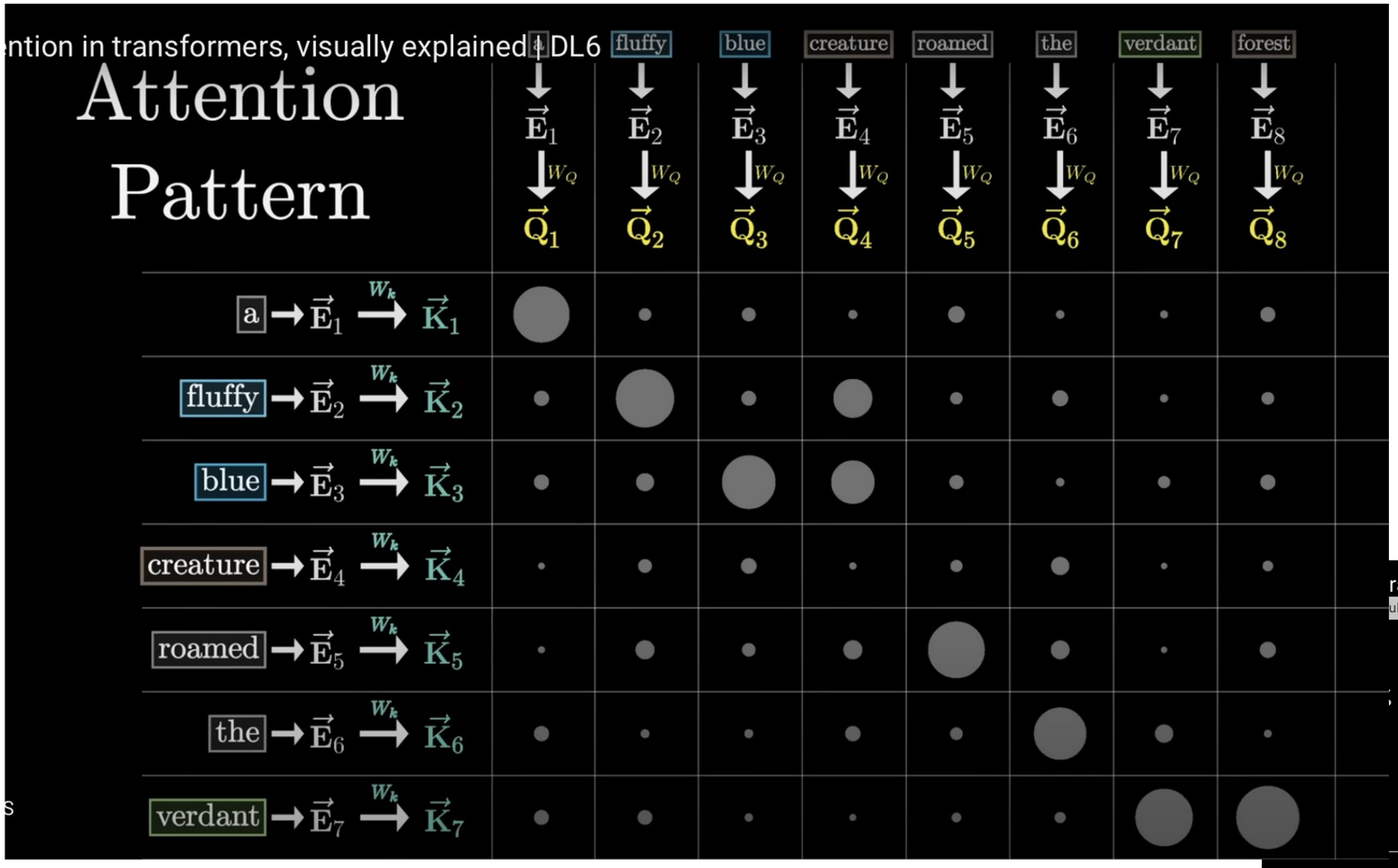


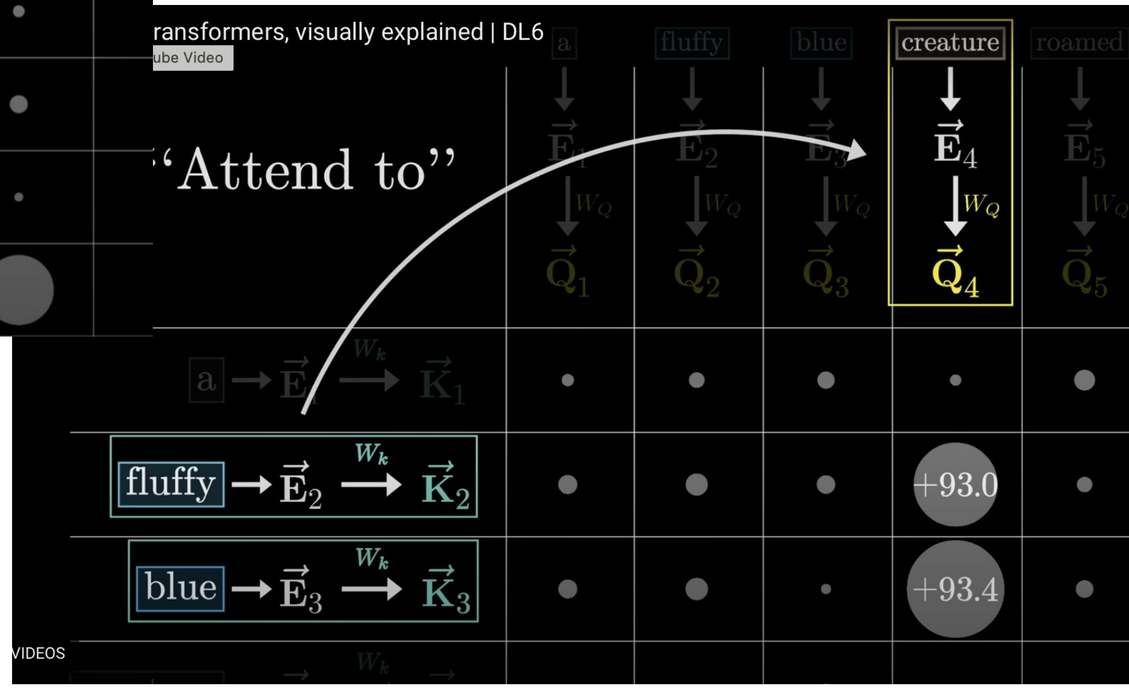




As an example consider the sentence above. You want the adjectives fluffy and blue to impact the word creature, and verdant to impact forest. In the image on the right, these are shown to have strong connections, or high attention values

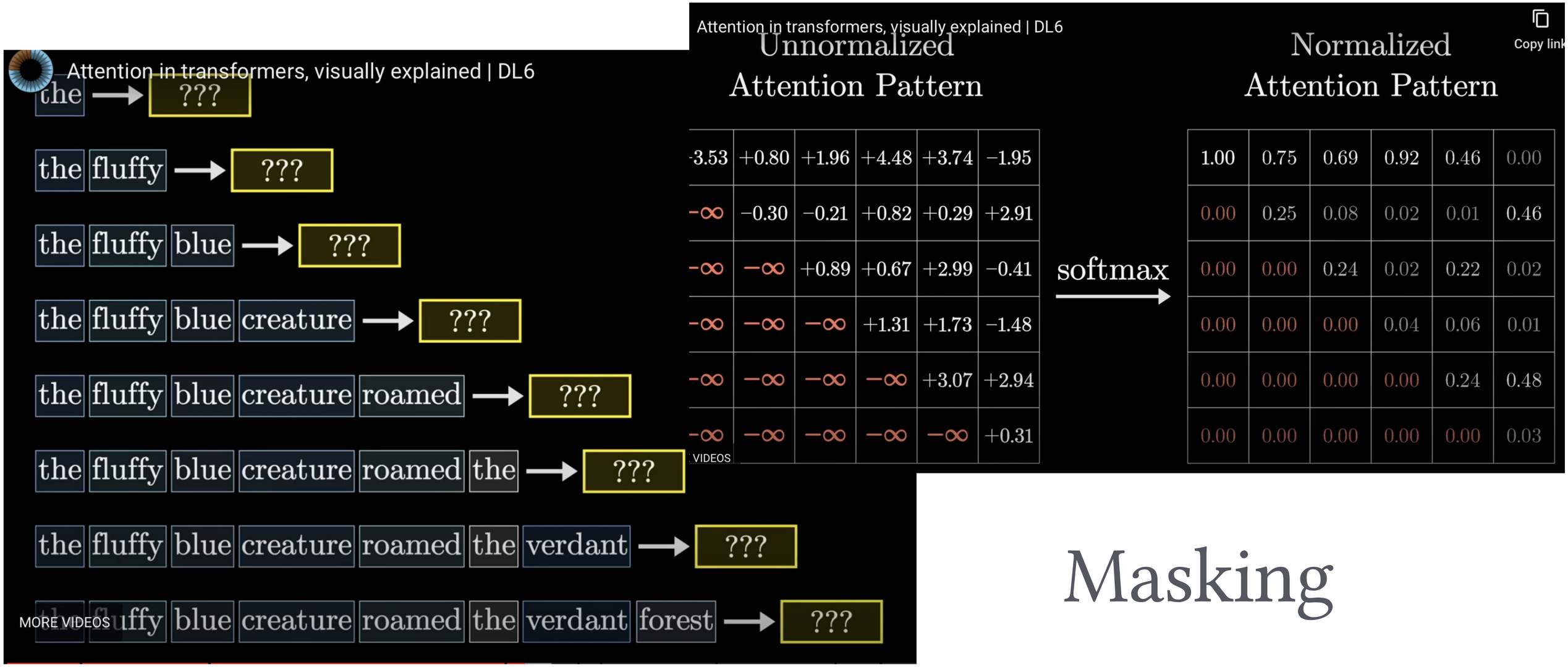






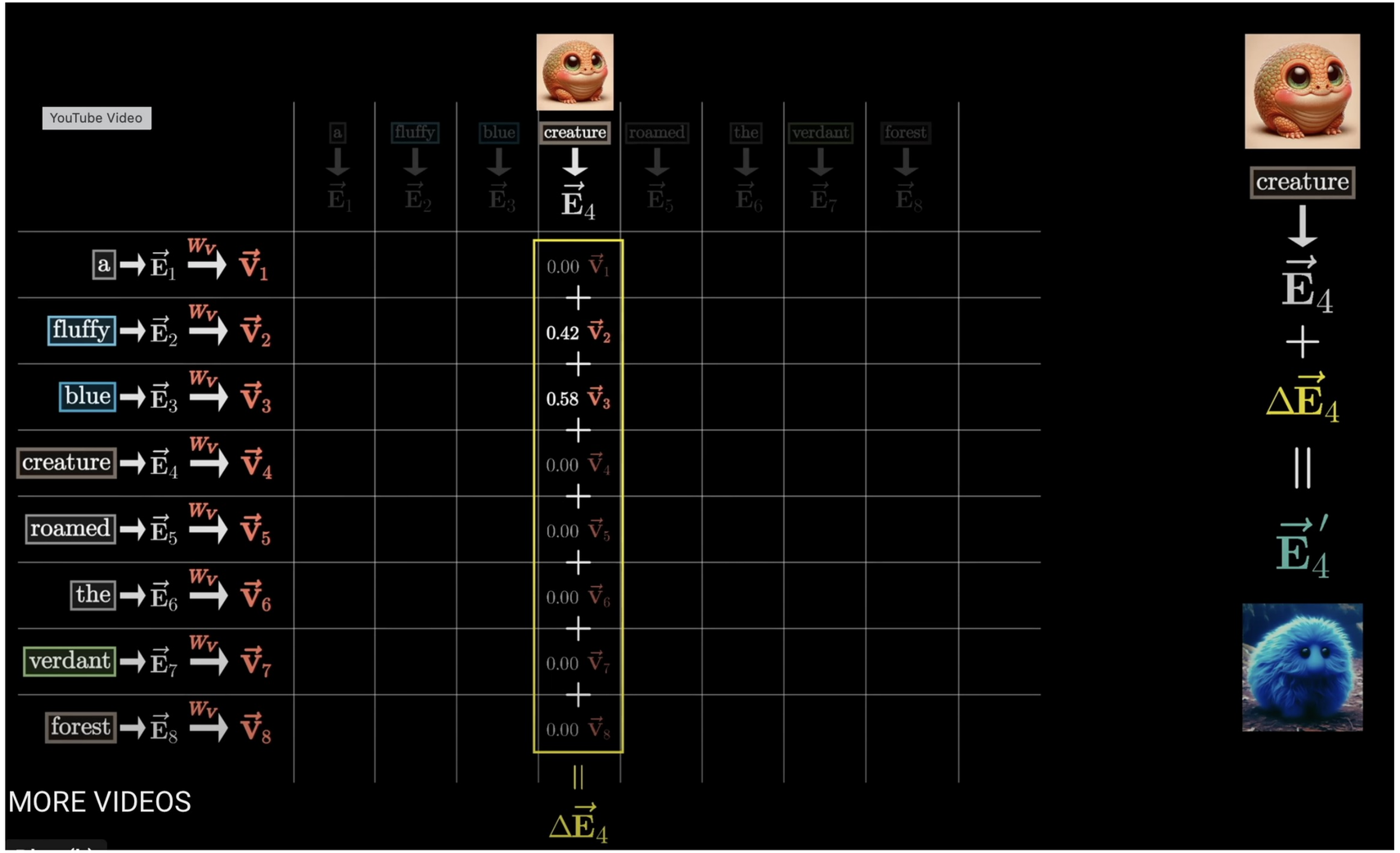
Attention

Dot Product



Masking

This is because we do not want future tokens in language to affect the past. So, for example, the only token hat can affect the is the. But creature can be impacted by the, fluffy, and blue. Setting the raw dot-product values to makes sure that on softmax, these go to 0, and the normalization is over the other tokens.



Now we multiply the for each of the context tokens by its attention based relevance score to combine the information from all the context in proportion to what is more important

Copy link

Multi-headed attention

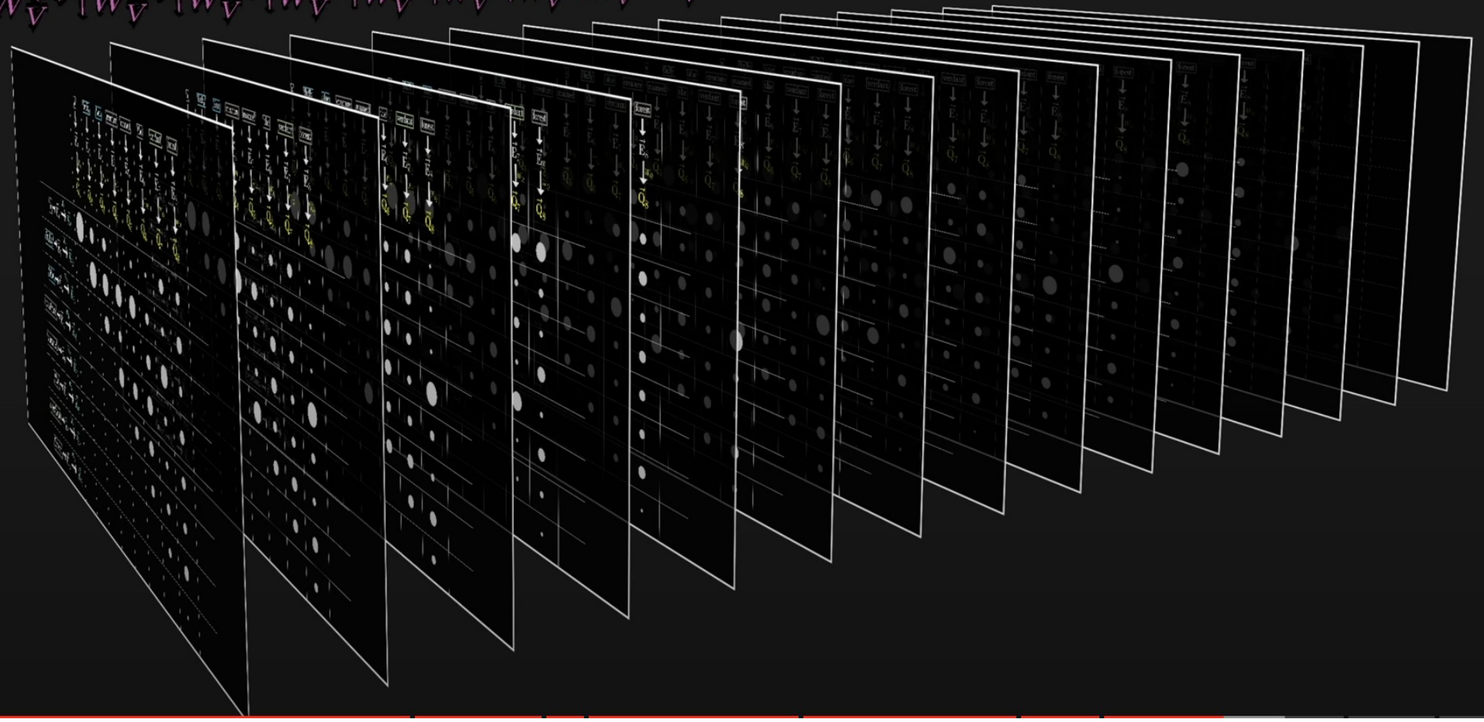
Video

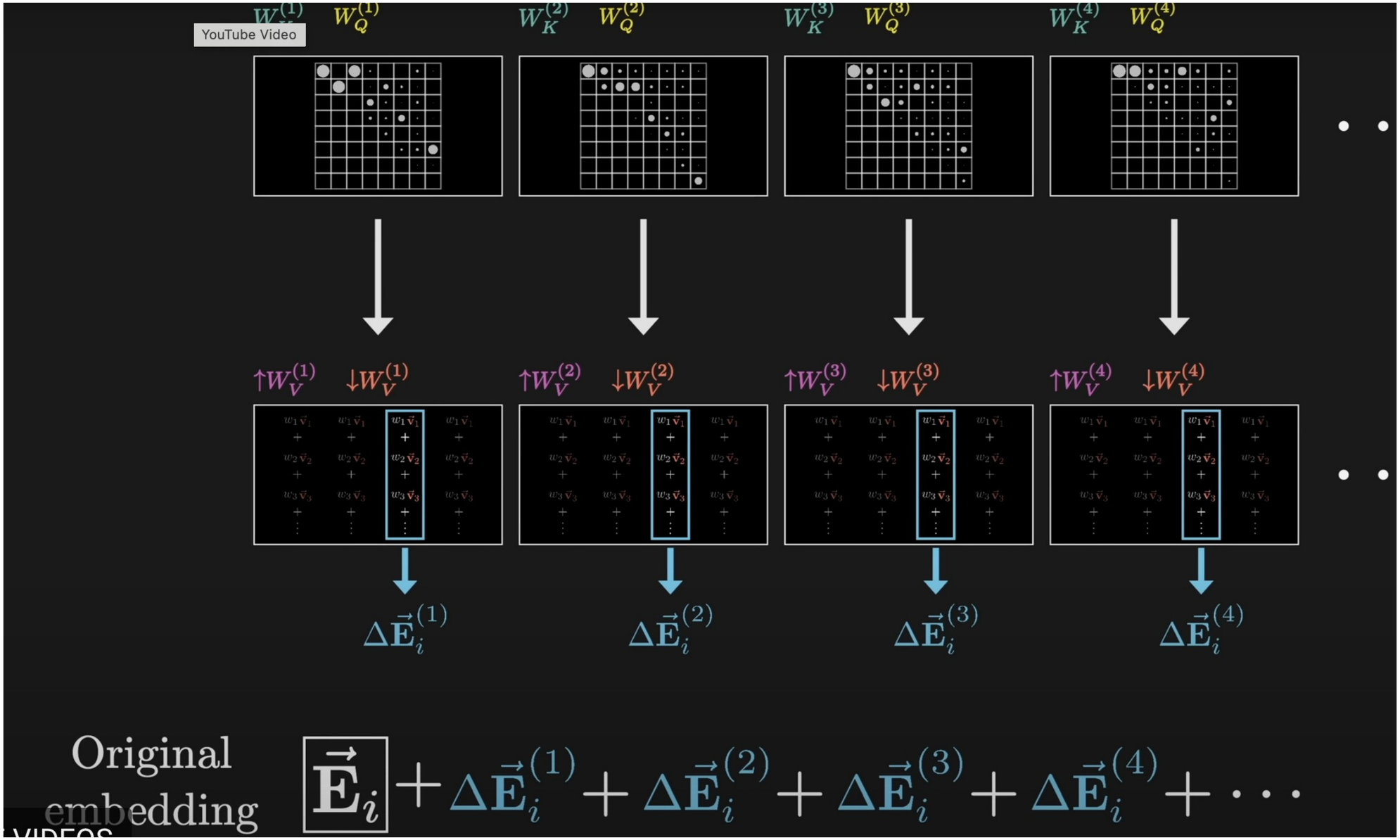
$$W_Q^{(1)}\ W_Q^{(2)}\ W_Q^{(3)}\ W_Q^{(4)}\ W_Q^{(5)}\ W_Q^{(6)}\ W_Q^{(7)}\ W_Q^{(8)}\ W_Q^{(9)}\ \cdots$$

$$W_K^{(1)}\ W_K^{(2)}\ W_K^{(3)}\ W_K^{(4)}\ W_K^{(5)}\ W_K^{(6)}\ W_K^{(7)}\ W_K^{(8)}\ W_K^{(9)}\ \cdots$$

$$\downarrow W_V^{(1)}\ \downarrow W_V^{(2)}\ \downarrow W_V^{(3)}\ \downarrow W_V^{(4)}\ \downarrow W_V^{(5)}\ \downarrow W_V^{(6)}\ \downarrow W_V^{(7)}\ \downarrow W_V^{(8)}\ \downarrow W_V^{(9)}\ \cdots$$

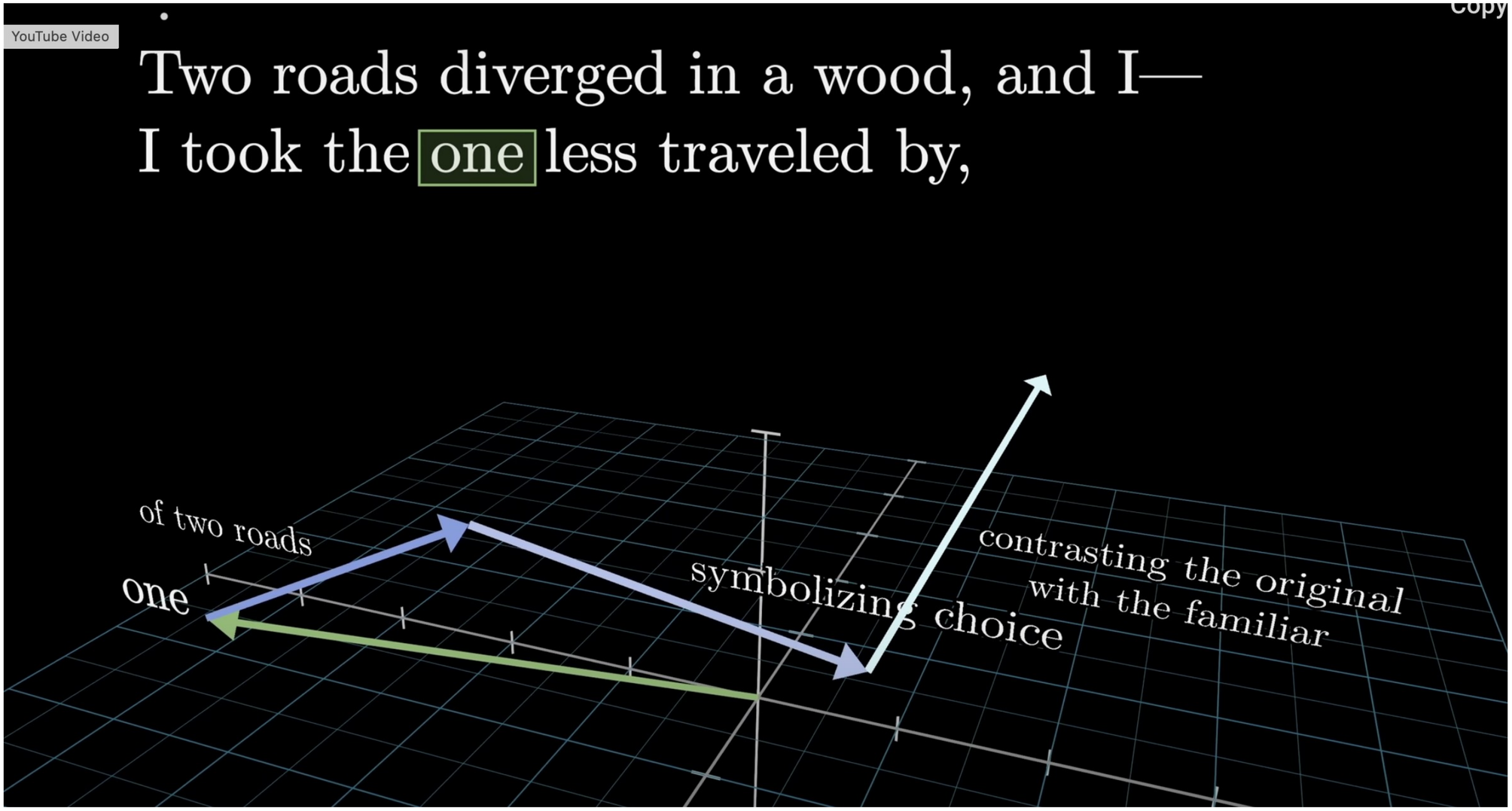
$$\uparrow W_V^{(1)}\ \uparrow W_V^{(2)}\ \uparrow W_V^{(3)}\ \uparrow W_V^{(4)}\ \uparrow W_V^{(5)}\ \uparrow W_V^{(6)}\ \uparrow W_V^{(7)}\ \uparrow W_V^{(8)}\ \uparrow W_V^{(9)}\ \cdots$$

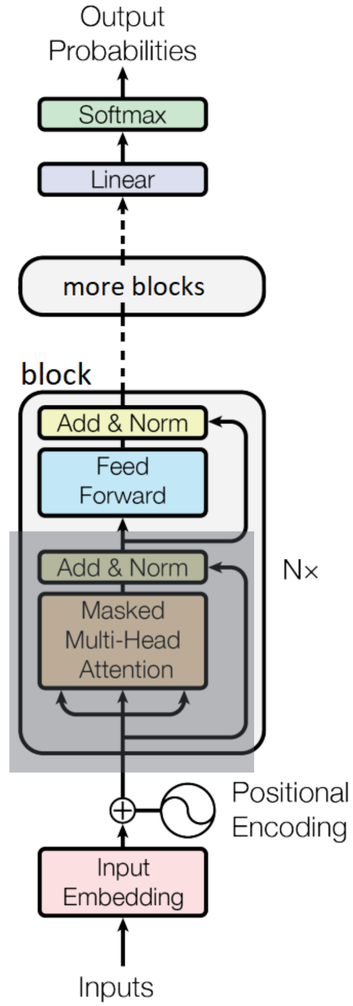




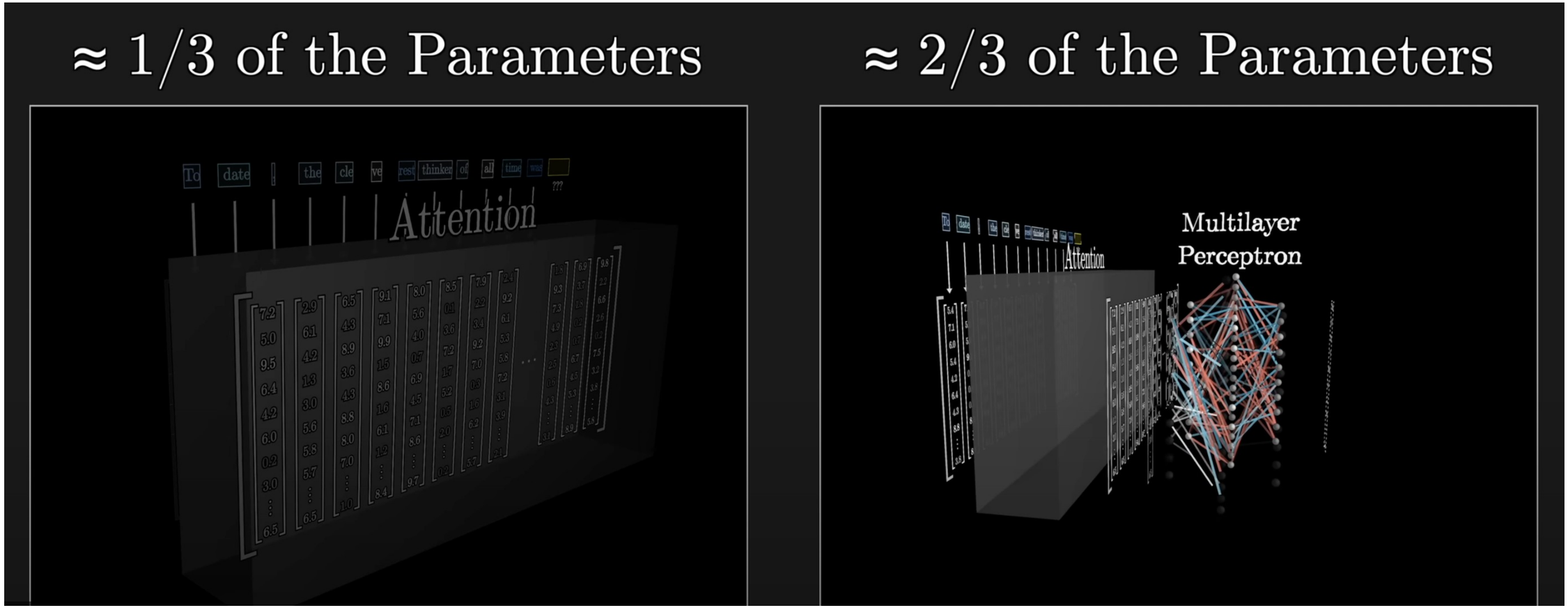
Now, in parallel, we calculate the nudge for each and every token

Remember: we are always adding to the original vector to nudge it

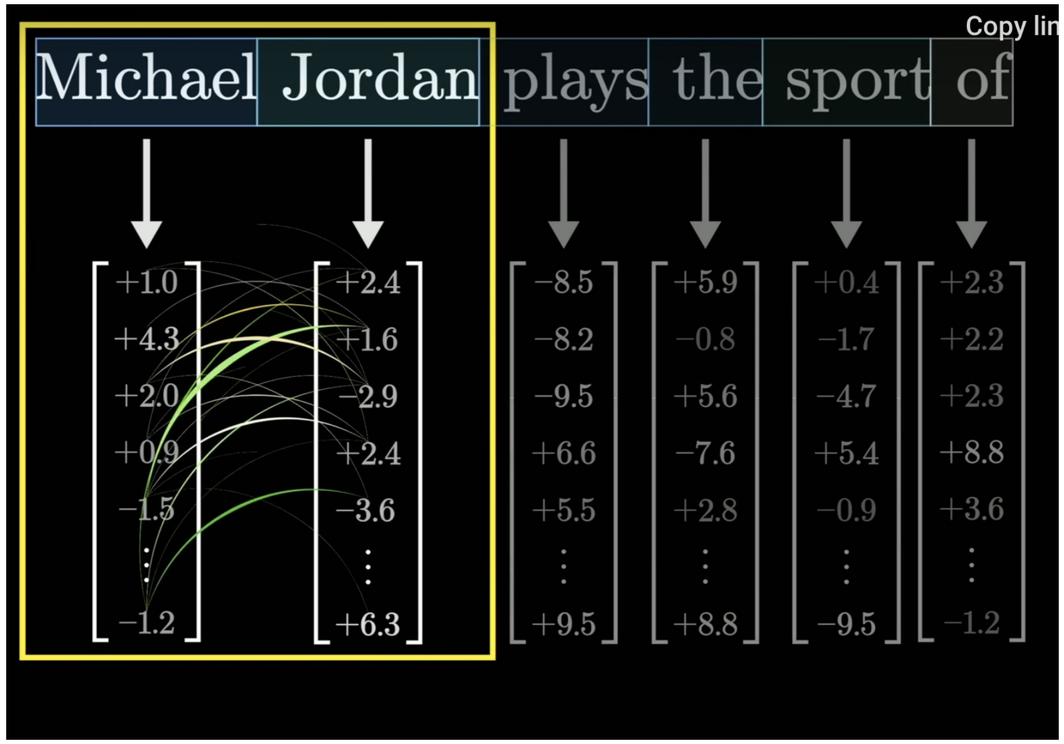


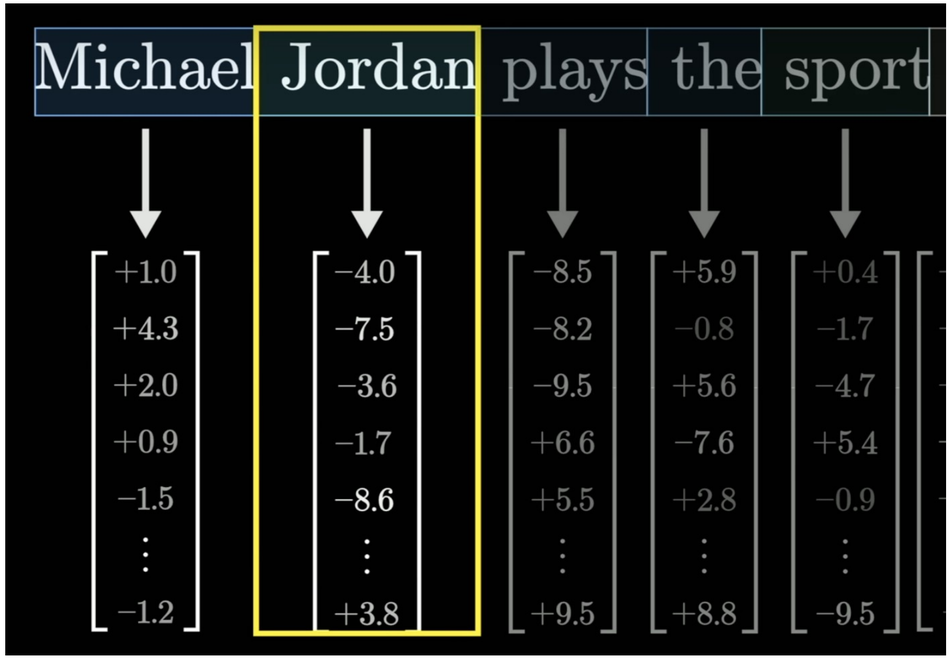


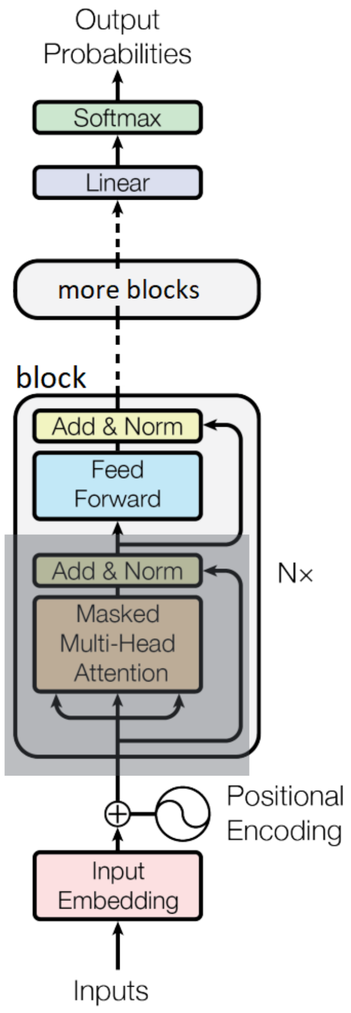
MLP provides the memory

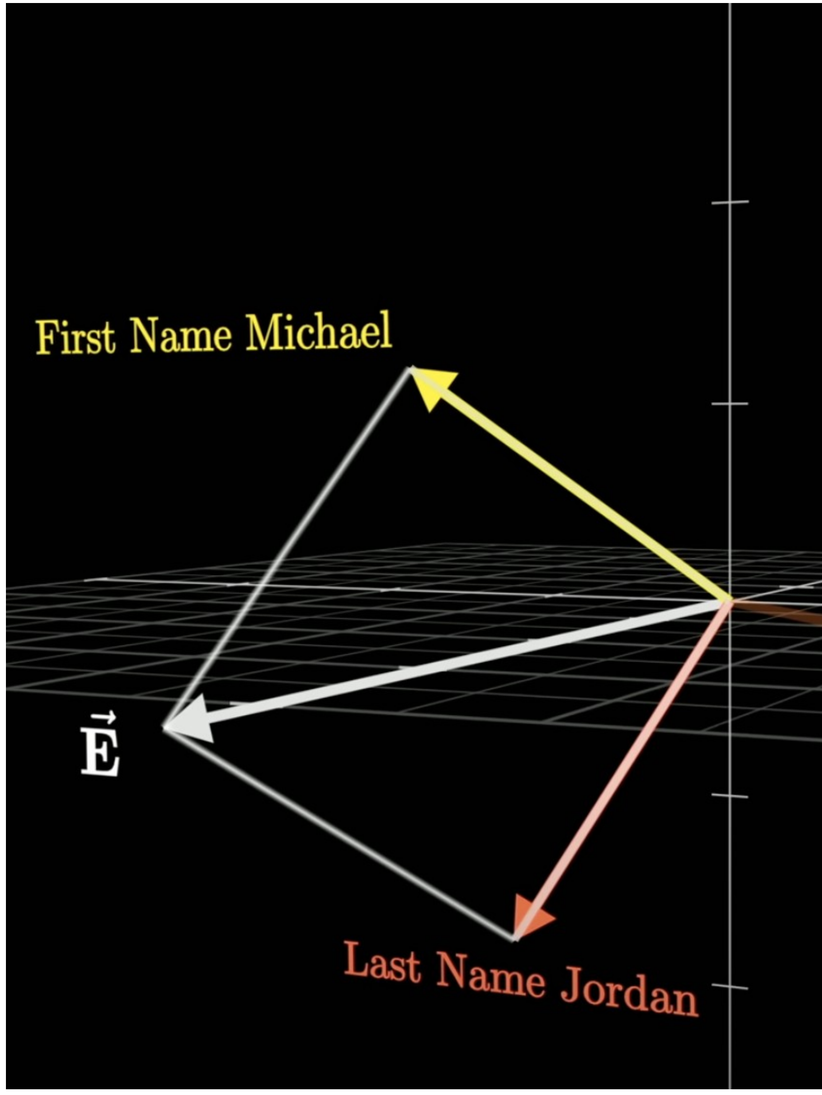


Now that we have the context-nudged token, we want to see what equipping our model with a memory does to that token.



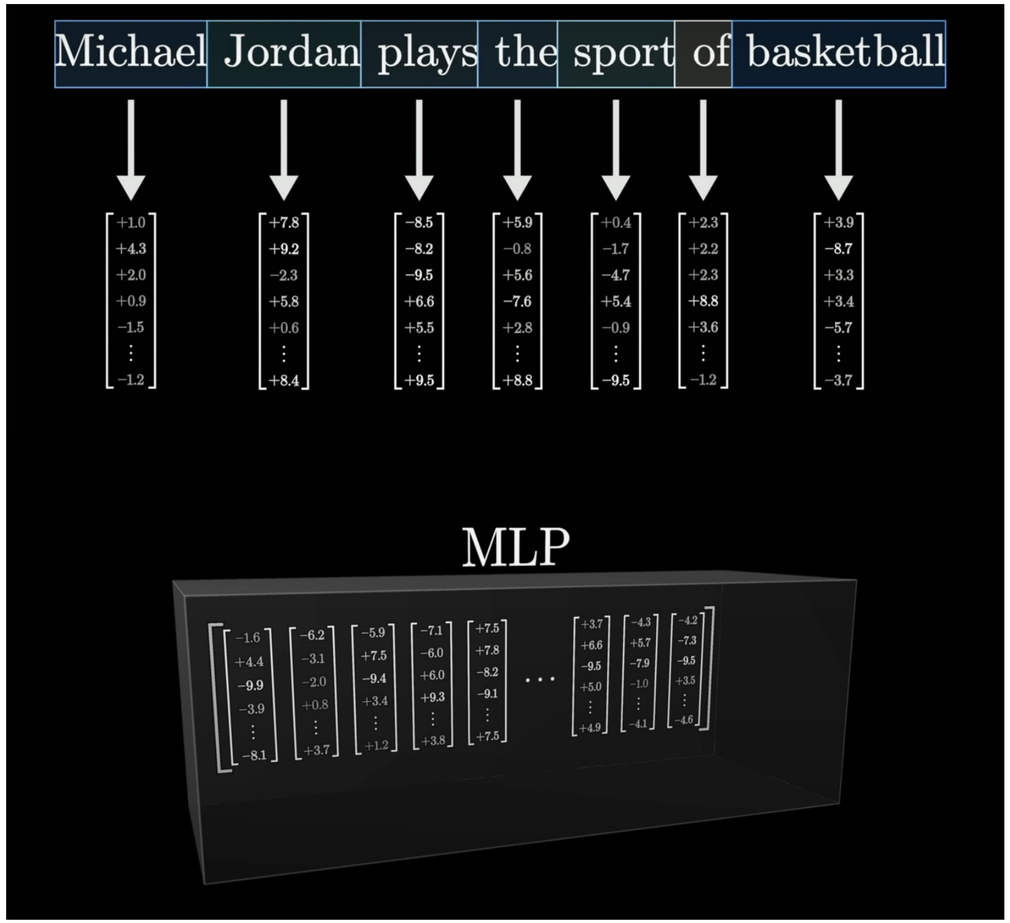


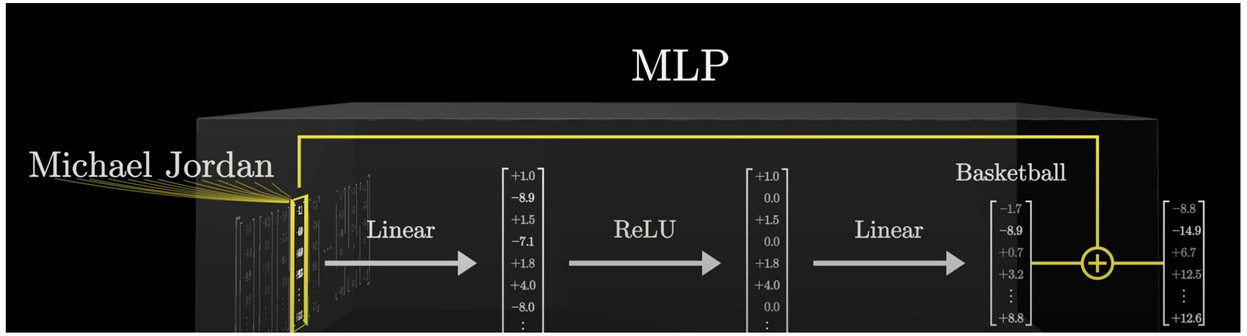


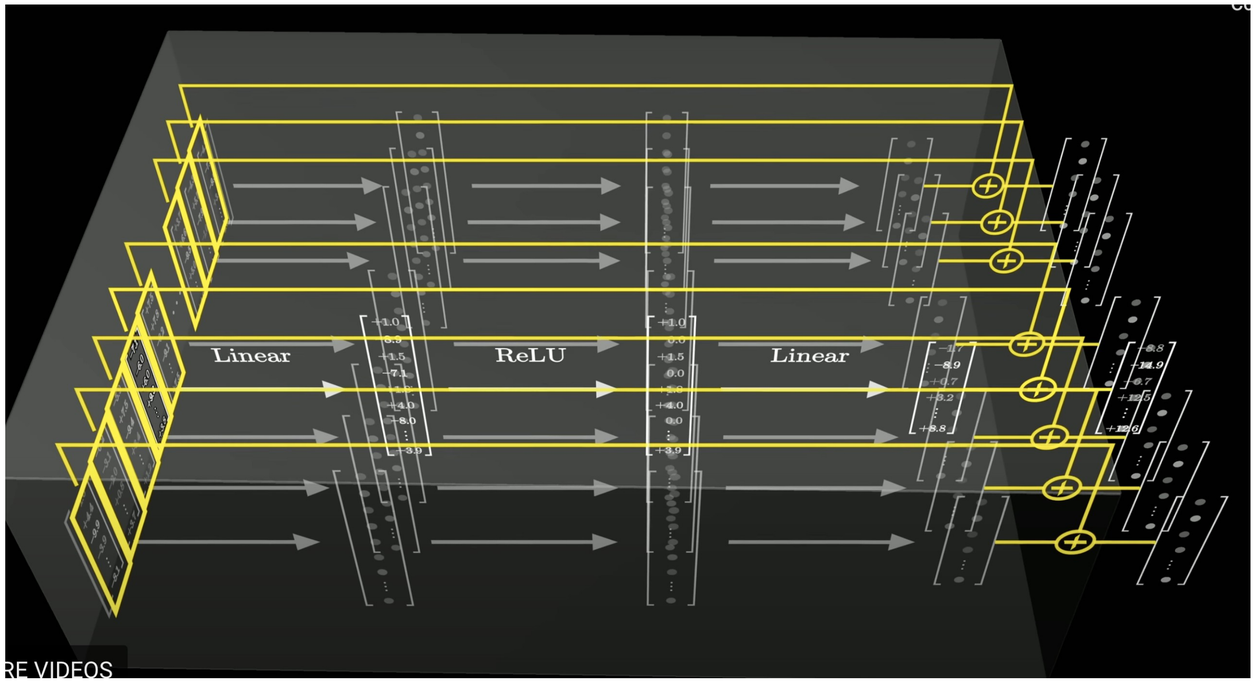


Lets say we want to complete the sentence “Michael Jordan plays the sport of …”. Lets say we went to the attention process, and nudged the embedding of “Jordan”by something that captures the first name “Michael”

Structure of the MLP



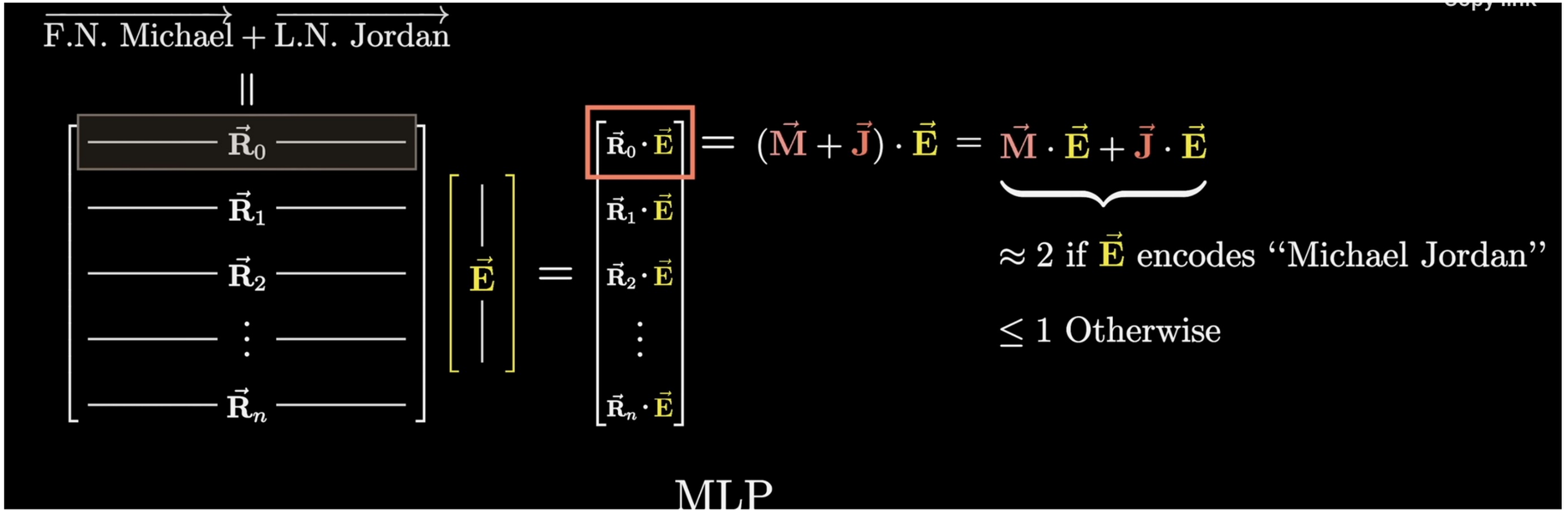


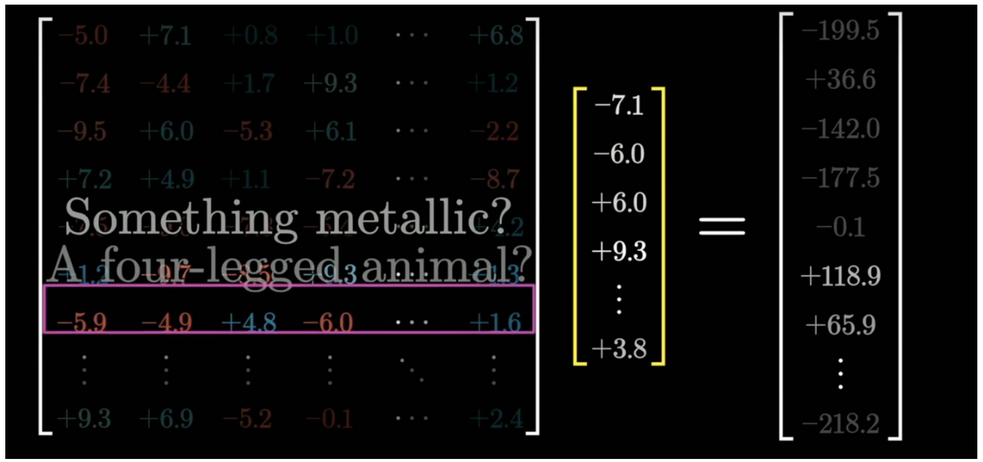


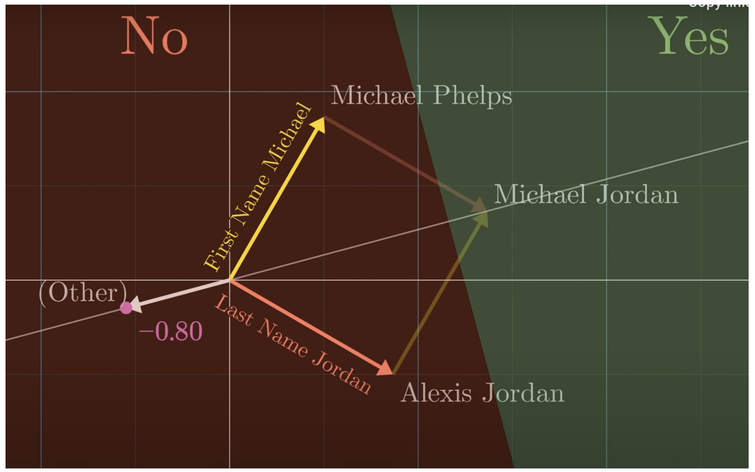
Our MLP works in parallel on the attention processed embeddings. We want it to take the “Michael Jordan” associated at token 2 and have that recall the token basketball somehow. We have an up-projection Linear layer, followed by a ReLU, and then a Linear down projection, back to embedding size.

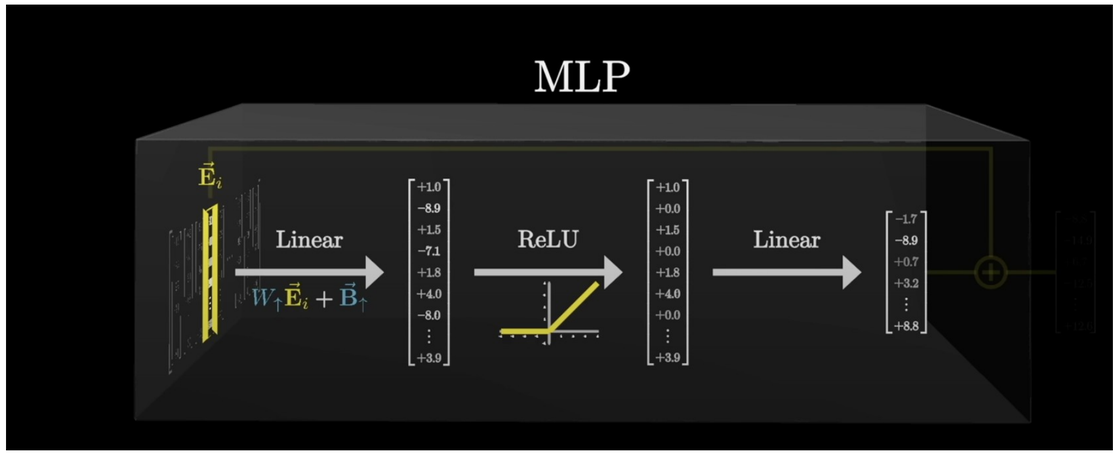
The lookup key comes from the first layer

Imagine the first row of the up-projection matrix codes Michael and Jordan. Then its dot product with the attention-modified embedding at Jordan will be very high. Now we do a ReLU. This will eliminate all negative dot products, thus removing other Michaels and Jordans from consideration. A bias term helps with this. The other rows may encode other concepts in the memory as shown to the right.









So far we have done an up-projection and ReLU as seen to the left. Now we must down-project back to the size of the embedding as shown below. How do we associate "Michael Jordan" with Basketball using this down-projection?

"Down projection"

$$\begin{bmatrix} +0.5 & +8.4 & -4.7 & -8.6 & +4.7 & +5.4 & +8.1 & \cdots & -9.6 \\ -5.3 & +2.3 & +8.9 & +8.9 & +1.1 & +8.2 & +2.8 & \cdots & -0.3 \\ +2.1 & +1.0 & +8.4 & +8.3 & -2.1 & +9.2 & -6.5 & \cdots & -7.2 \\ +0.1 & -9.5 & +8.9 & +6.5 & -9.6 & -6.4 & -3.3 & \cdots & +6.1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ -4.2 & -0.2 & +2.0 & -9.6 & +1.9 & -1.3 & +6.1 & \cdots & +7.8 \end{bmatrix} \begin{bmatrix} +1.0 \\ +0.0 \\ +1.5 \\ +0.0 \\ +1.8 \\ +4.0 \\ +0.0 \\ \vdots \\ +3.9 \end{bmatrix} + \begin{bmatrix} +1.5 \\ -6.3 \\ +5.7 \\ +2.2 \\ \vdots \\ -1.6 \end{bmatrix} = \begin{bmatrix} -1.7 \\ -8.9 \\ +0.7 \\ +3.2 \\ \vdots \\ +8.8 \end{bmatrix} \Bigg\} 12{,}288$$

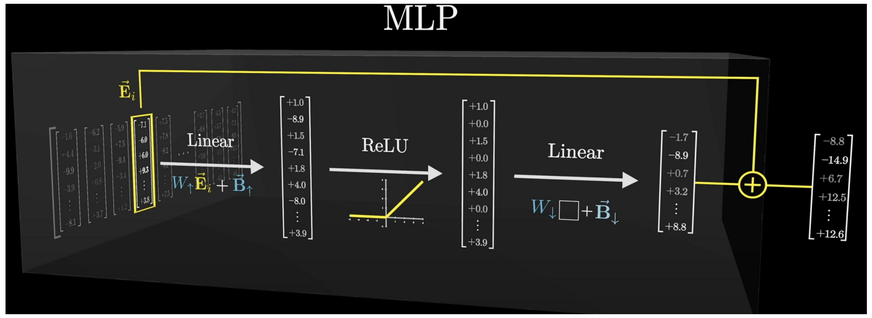
The lookup value comes from the next layer

$$n_0\vec{\mathbf{C}}_0 + n_1\vec{\mathbf{C}}_1 + \boxed{n_2\vec{\mathbf{C}}_2} + n_3\vec{\mathbf{C}}_3 + n_4\vec{\mathbf{C}}_4 + \cdots + n_m\vec{\mathbf{C}}_m$$

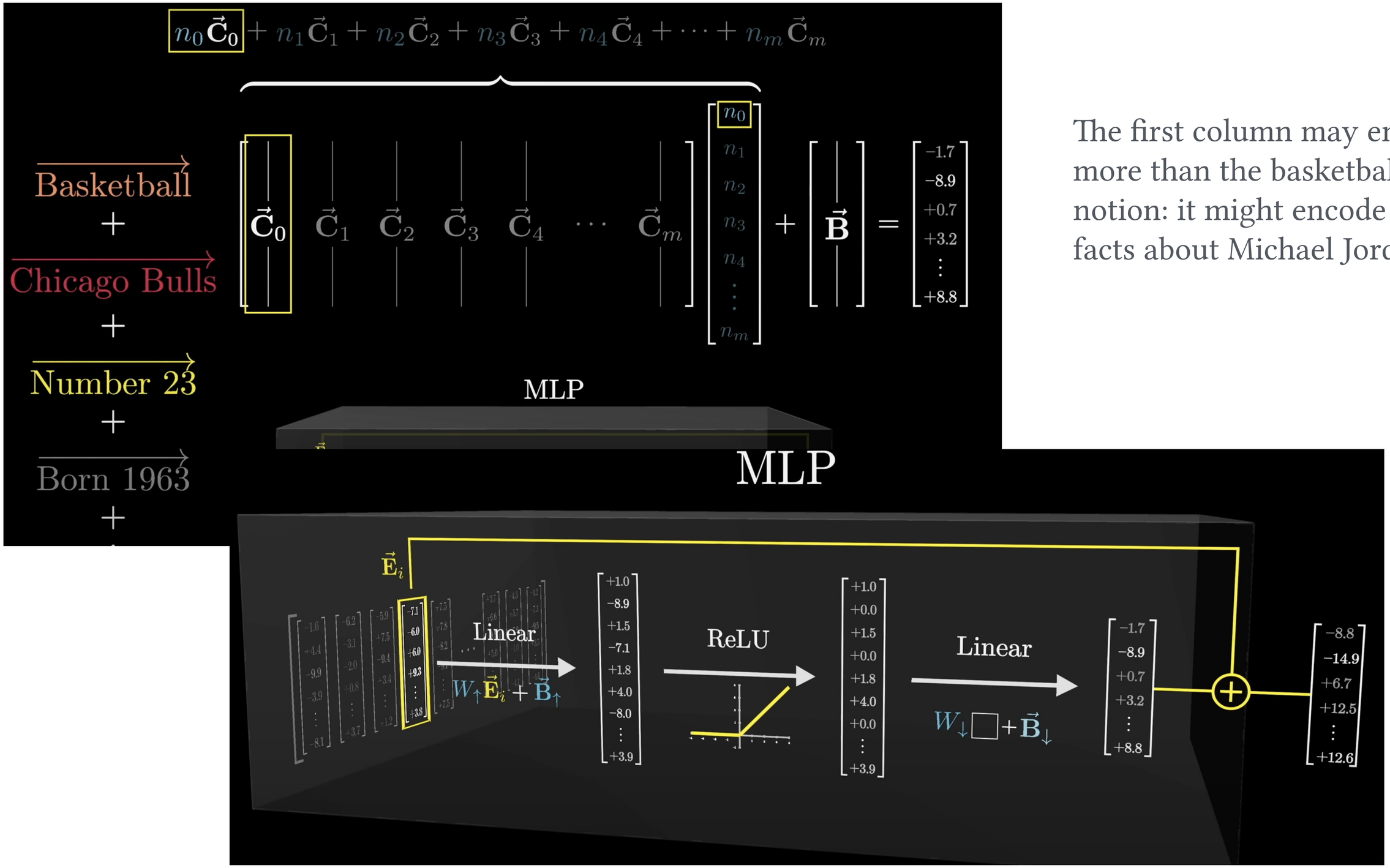
$$\begin{bmatrix} | & | & | & | & | & & | \\ \vec{\mathbf{C}}_0 & \vec{\mathbf{C}}_1 & \vec{\mathbf{C}}_2 & \vec{\mathbf{C}}_3 & \vec{\mathbf{C}}_4 & \cdots & \vec{\mathbf{C}}_m \\ | & | & | & | & | & & | \end{bmatrix} \begin{bmatrix} n_0 \\ n_1 \\ n_2 \\ n_3 \\ n_4 \\ \vdots \\ n_m \end{bmatrix} + \begin{bmatrix} | \\ \vec{\mathbf{B}} \\ | \end{bmatrix} = \begin{bmatrix} -1.7 \\ -8.9 \\ +0.7 \\ +3.2 \\ \vdots \\ +8.8 \end{bmatrix}$$

basketball

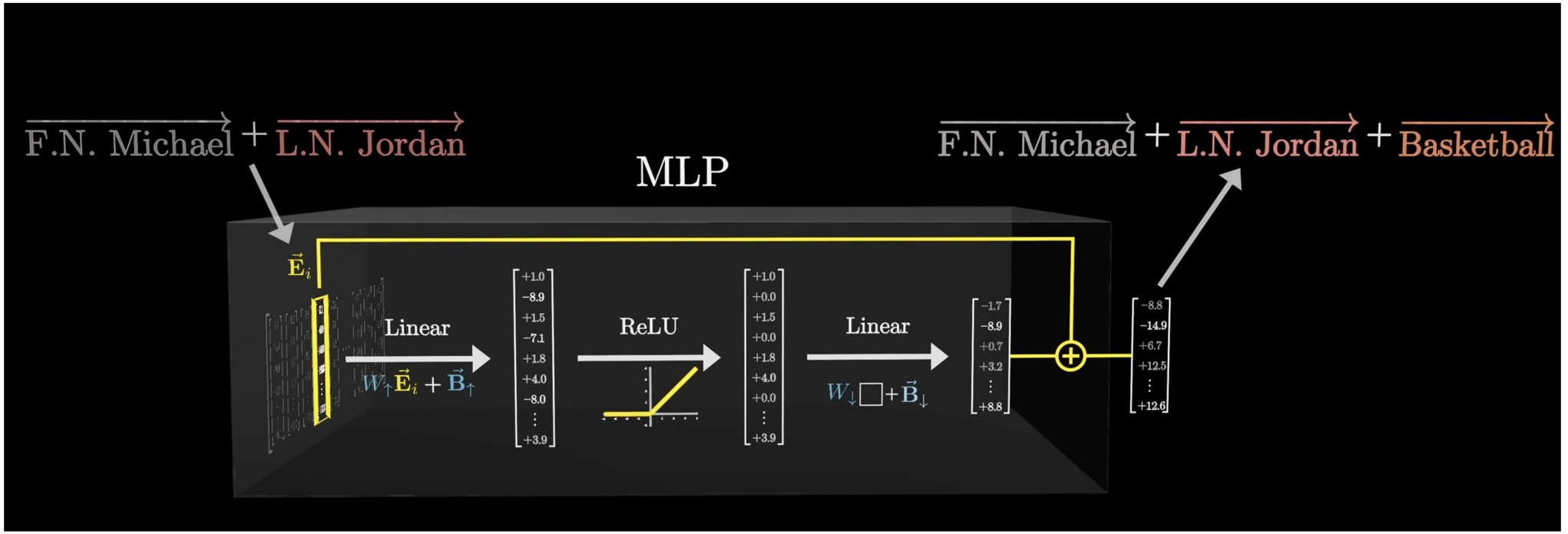
The trick is to notice that matrix multiplication can also be thought of as a weighted sum of columns.

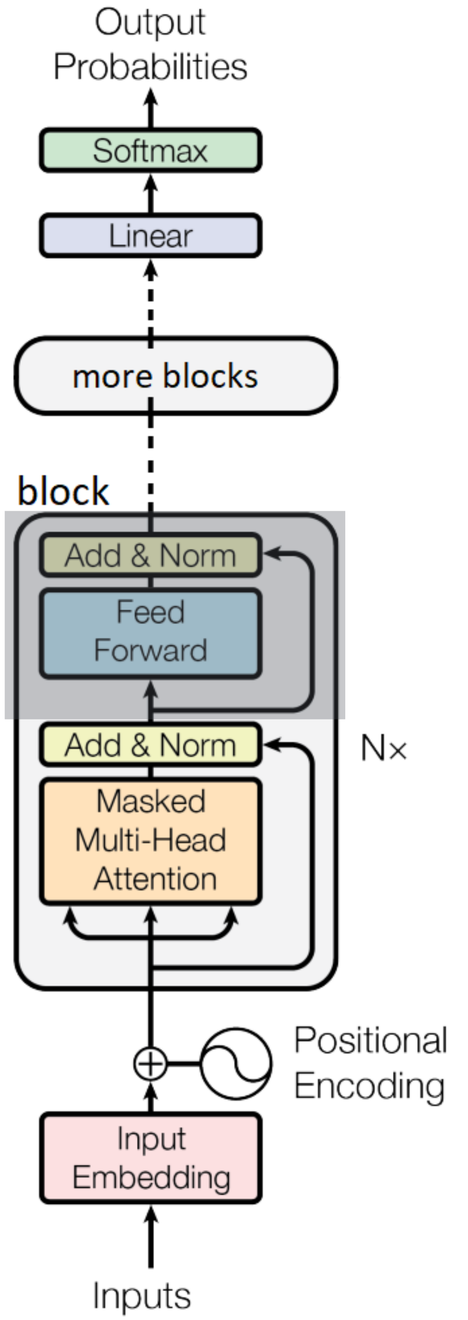


Recall that the first entry of the up-projected vector is large because the first column of the up-projection matrix encoded “Michael Jordan”. Thus the first column will have an outsized influence on the finally calculated embedding nudge (its a nudge because we are adding this effect to the attention-processed embedding). If that first column encodes “basketball”, the values from the first column will be preferentially picked out.

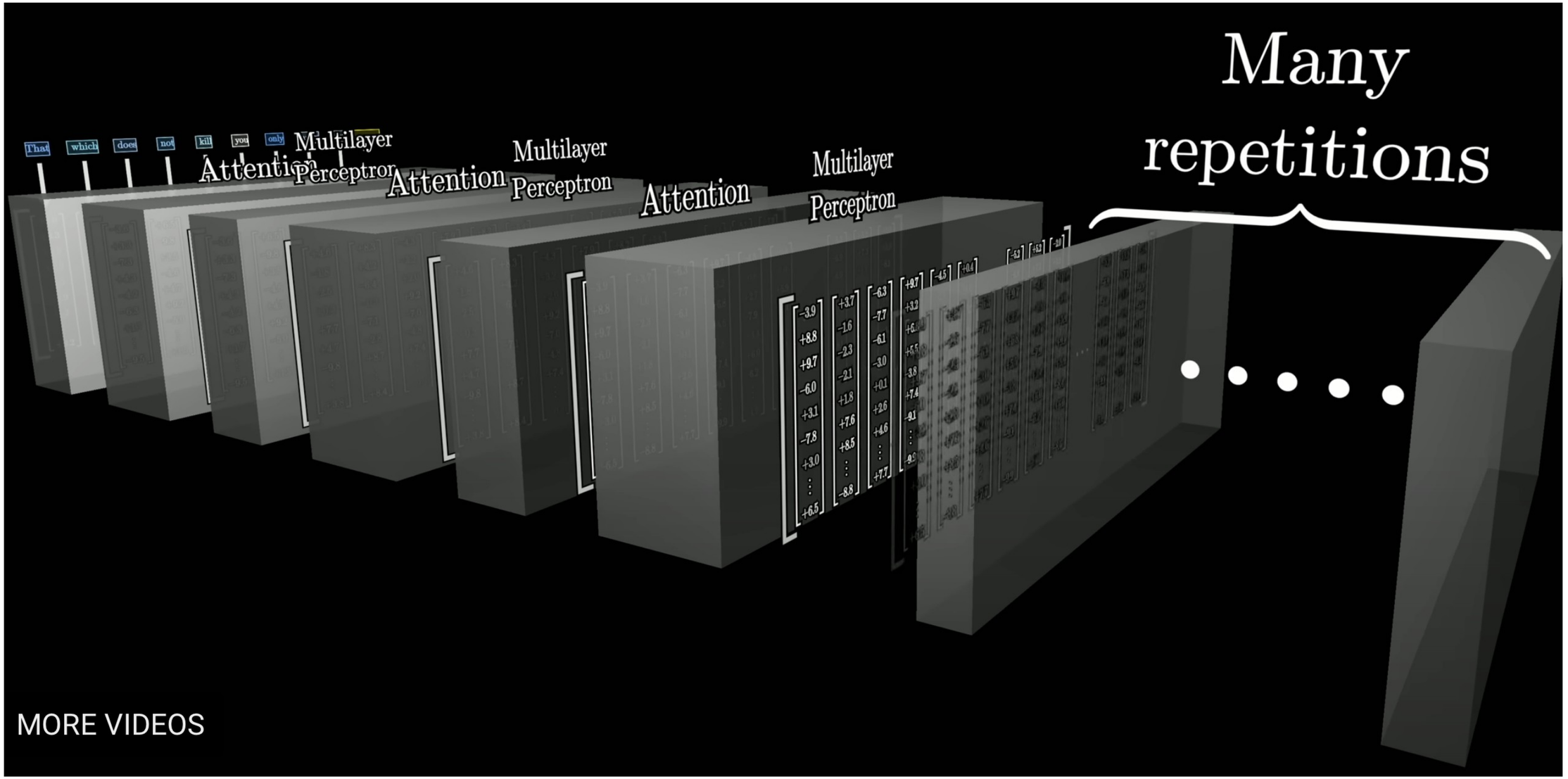


The first column may encode more than the basketball notion: it might encode more facts about Michael Jordan





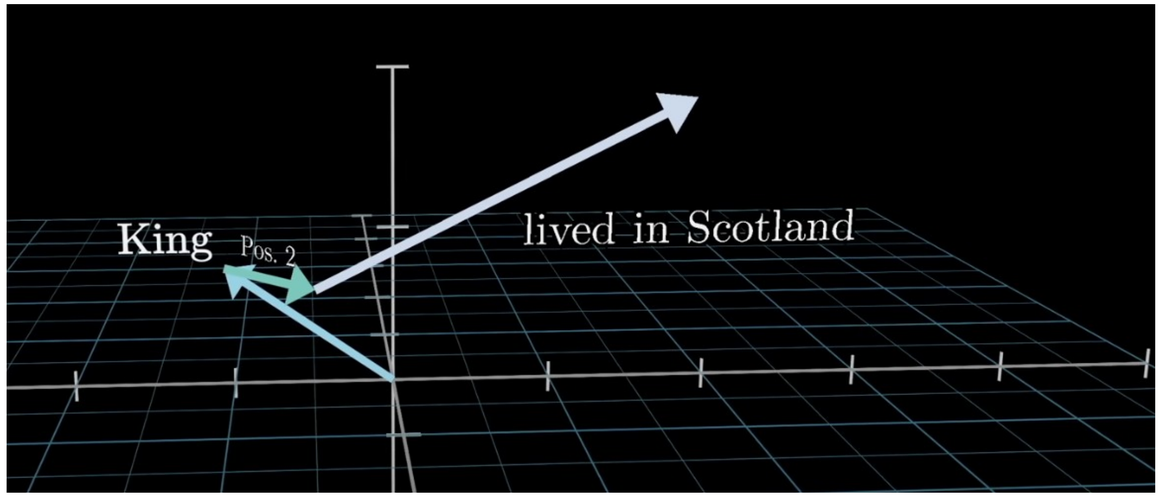
We started with “Jordan” at the second token in the sequence. The attention mechanism added the “Michael” vector onto it. Now the memory mechanism adds the “basketball” o to it, so that the “Jordan” embedding has been nudged into both having Michael picked u via the attention mechanism and basketball picked up via the memory mechanism.



The process of context and memory repeats many times in the transformer, allowing multiple contaxts and memories to be added. This is how gpt like models can seem to be reasoning: they are building in a lot of memory, and the interaction of multiple contexts with this memory seems like they can interpolate between multiple knowledge bases,

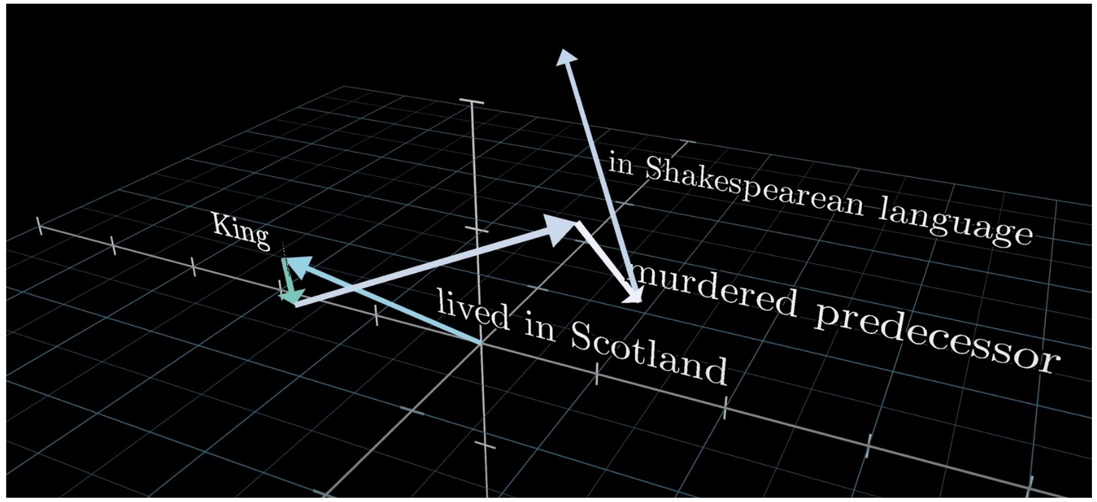
What are we trying to do with embeddings?

The King doth wake tonight and takes his rouse ...



1. We want information to be picked from the current context: drinking, excesses

ATTENTION



2. We want information to come with a memory: Hamlet, Shakespeare, Claudius

MLP

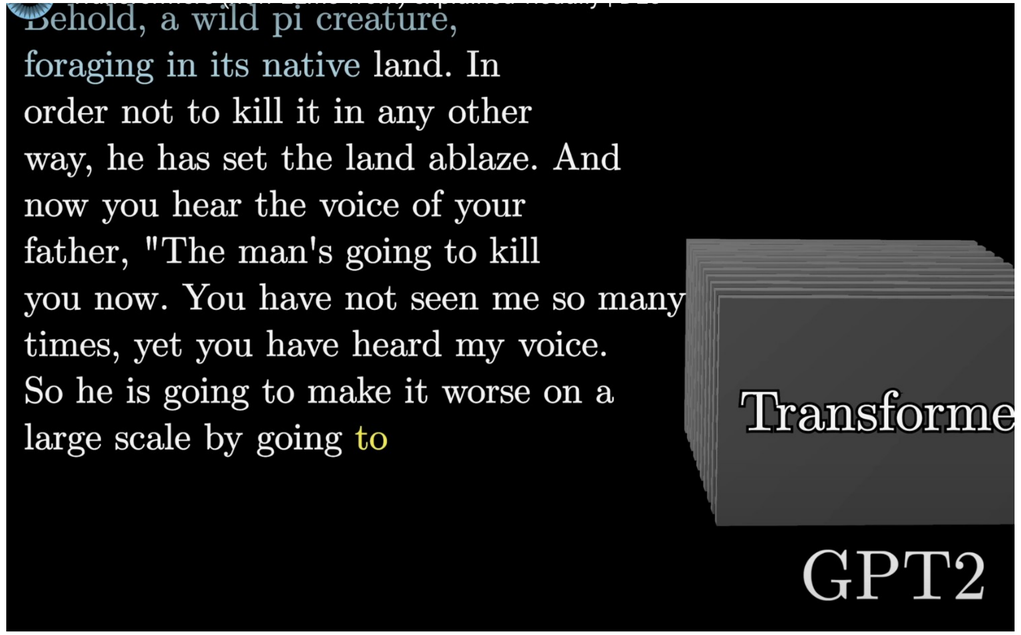
3. We want information to come from multiple contexts.

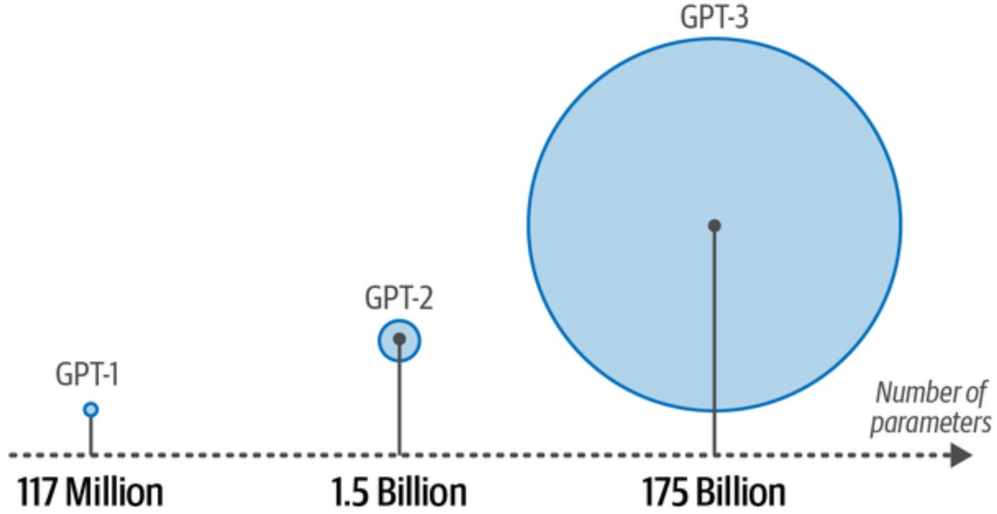
MULTIPLE HEADS

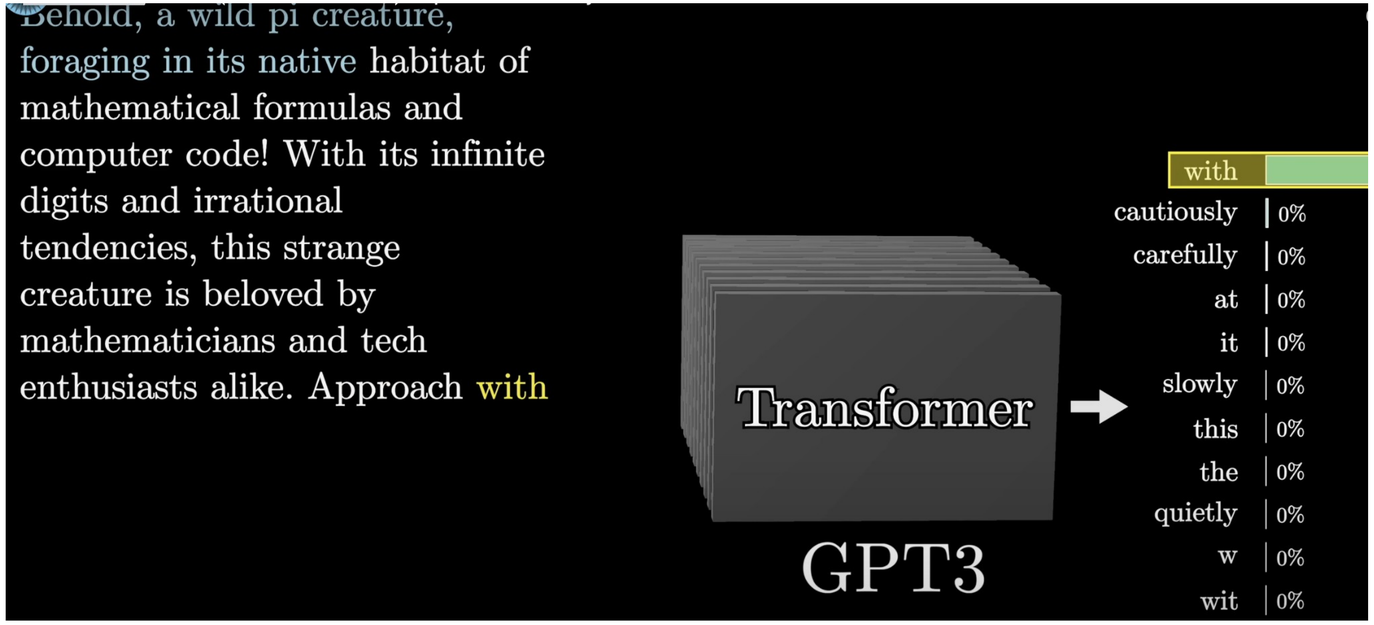
Intelligence?

And so, between the attention and MLP layers, a transformer forms a building block that entwines context and memory, and gives us the notion of an intelligent being interpolating between different gobs of memory in the context of the current discussion. This explains why gpt models seem intelligent, and also hints at why they go off the rails with hallucinations at times.

From GPT-2 to GPT-3







The history of language modeling

